



**Pontificia Universidad
Católica del Ecuador**
Seréis mis testigos

MANABÍ

**PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR
SEDE MANABÍ**

CARRERA DE SOFTWARE

TRABAJO DE TITULACIÓN

**DESARROLLO DE UNA PLATAFORMA SAAS QUE INTEGRA
MODULOS CON IA PARA LA GESTIÓN INTEGRAL DE PYMES
GASTRONÓMICAS EN PORTOVIEJO**

LÍNEA DE INVESTIGACIÓN

**INGENIERÍA DE SOFTWARE Y SISTEMAS COMPUTACIONALES PARA LA
TRANSFORMACIÓN DIGITAL EMPRESARIAL**

SUBLÍNEA DE INVESTIGACIÓN

**TRANSFORMACIÓN DIGITAL DE PYMES Y SISTEMAS SAAS
PROCESOS EMPRESARIALES**

**PREVIO AL TÍTULO DE
INGENIERO EN SOFTWARE**

AUTORES

**LUIGGI ANDREE ARTEAGA SANTANA
CHRISTIAN ORLANDO SARAGURO ENCALADA**

TUTOR

JOSE NARANJO, M.ENG.

PORTOVIEJO, FEBRERO 2026

Certificación del Tutor

En mi calidad de tutor del trabajo de integración curricular, certifico haber revisado el presente manuscrito de investigación, el mismo que se ajusta a las normas vigentes de la Pontificia Universidad Católica del Ecuador Sede Manabí, cumpliendo la Normativa del Trabajo de Integración Curricular; en consecuencia, es apto para su presentación y sustentación. En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

JOSE NARANJO, M.ENG.

CI: 1003528674

Tutor(a)

2026

Acta de aprobación del Tribunal

El jurado examinador aprueba el presente trabajo de integración curricular en nombre de la Pontificia Universidad Católica del Ecuador, Sede Manabí.

En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

Nombre del revisor 1

CI:0000000000

Lector Revisor1(a)

Nombre del revisor 2

CI:0000000000

Lector Revisor2(a)

JOSE NARANJO, M.ENG

CI: 1003528674

Tutor(a)

2026

Declaración de Originalidad

Este manuscrito no contiene ningún tipo de material que ha sido aceptado para la obtención de un título universitario en otra institución, excepto en forma de información de soporte que ha sido debidamente citada en mi trabajo. Este trabajo es de total responsabilidad del autor, quien declara bajo juramento que ninguna sección de este trabajo de integración curricular infringe los derechos de autor de nadie.

En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

LUIGGI ARTEAGA

C.I.: 1314693597

Autor

CHRISTIAN SARAGURO

C.I.: 1350274484

Autor

2026

Declaración de Derecho del Autor

Autorizo a la Pontificia Universidad Católica del Ecuador a distribuir este manuscrito de investigación en medios físicos y electrónicos con el fin de promover la divulgación de mis resultados a la comunidad científica y a la sociedad en general. Adicionalmente autorizo el uso de los contenidos de esta investigación como bibliografía para fines académicos, por cualquier medio o procedimiento, citando como fuente de información al autor de este trabajo. En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

LUIGGI ARTEAGA

C.I.: 1314693597

Autor

CHRISTIAN SARAGURO

C.I.: 1350274484

Autor

2026

Dedicatoria

Dedico este trabajo, en primer lugar, a Dios, por ser mi fortaleza en los momentos difíciles y por permitirme alcanzar una de las metas más importantes de mi vida.

A mi familia, por su amor incondicional, sacrificio, paciencia y por creer siempre en mí, incluso cuando yo mismo dudaba. Su apoyo constante ha sido el motor que me impulsó a seguir adelante.

A mis padres, por ser ejemplo de esfuerzo, responsabilidad y perseverancia, y por enseñarme que los sueños se alcanzan con trabajo y dedicación.

Finalmente, dedico este logro a todas aquellas personas que con una palabra de aliento, un consejo o un gesto de apoyo hicieron posible la culminación de esta etapa tan significativa en mi vida.

Agradecimientos

Agradezco profundamente a Dios por brindarme la fortaleza, salud y sabiduría necesarias para culminar esta importante etapa de mi formación profesional.

Mi sincero agradecimiento a mi familia, quienes han sido mi mayor apoyo y motivación constante, brindándome su amor, comprensión y respaldo incondicional en cada paso de este proceso.

Agradezco de manera especial a mis docentes, quienes con su conocimiento, paciencia y orientación contribuyeron significativamente a mi desarrollo académico y profesional, permitiéndome adquirir las herramientas necesarias para la realización de este trabajo.

Un agradecimiento particular a mi tutor/a de tesis, por su guía, tiempo y valiosos aportes durante el desarrollo de esta investigación, siendo un pilar fundamental para su culminación exitosa.

Finalmente, agradezco a todas las personas que, de manera directa o indirecta, colaboraron en la realización de este proyecto y formaron parte de este importante logro personal y profesional.

Resumen

Este trabajo presenta el diseño, desarrollo y validación de una plataforma SaaS integral para la digitalización de PYMES gastronómicas en Portoviejo. El objetivo principal es optimizar los procesos operativos de gestión de pedidos, inventarios, delivery y atención al cliente mediante la implementación de módulos tecnológicos integrados y accesibles.

La propuesta consiste en una arquitectura multiplataforma que integra un sistema de gestión de pedidos con geolocalización en tiempo real, módulo de control de inventarios automatizado, panel administrativo con analítica de indicadores clave de desempeño (KPIs), y un chatbot basado en inteligencia artificial utilizando tecnología RAG (Retrieval-Augmented Generation) para atención automatizada. El desarrollo se realizó bajo metodología ágil Scrum, empleando tecnologías web modernas y bases de datos relacionales escalables.

Para evaluar la solución, se aplicó un enfoque metodológico mixto que combina encuestas estructuradas, entrevistas semiestructuradas y pruebas de usabilidad (métricas SUS) con usuarios reales de 2 PYMES seleccionadas por conveniencia. Los resultados esperados incluyen la demostración de mejoras en tiempos de respuesta operativa, reducción de errores en gestión de pedidos e inventarios, disminución de costos asociados a plataformas de terceros, y validación positiva de usabilidad y experiencia de usuario. Esta investigación se alinea con la Agenda de Transformación Digital del Ecuador 2022–2025 y contribuye al fortalecimiento competitivo del sector gastronómico de Portoviejo, reconocido por UNESCO como Ciudad Creativa de la Gastronomía.

Palabras clave: PYMES, gastronomía, digitalización, SaaS, Portoviejo.

Abstract

This work presents the design, development, and validation of an integrated SaaS platform for the digitalization of gastronomic SMEs in Portoviejo. The main objective is to optimize operational processes for order management, inventory, delivery, and customer service through the implementation of integrated and accessible technological modules.

The proposal consists of a multiplatform architecture that integrates an order management system with real-time geolocation, automated inventory control module, administrative dashboard with key performance indicators (KPIs) analytics, and an AI-based chatbot using RAG (Retrieval-Augmented Generation) technology for automated customer service. The development was carried out under the Scrum agile methodology, employing modern web technologies and scalable relational databases.

To evaluate the solution, a mixed methodological approach was applied, combining structured surveys, semi-structured interviews, and usability testing (SUS metrics) with real users from 2 SMEs selected by convenience. Expected results include demonstrating improvements in operational response times, reduction of errors in order and inventory management, decreased costs associated with third-party platforms, and positive validation of usability and user experience. This research aligns with Ecuador's Digital Transformation Agenda 2022–2025 and contributes to the competitive strengthening of Portoviejo's gastronomic sector, recognized by UNESCO as a Creative City of Gastronomy.

Keywords: SMEs, gastronomy, digitalization, SaaS, Portoviejo.

Tabla de Contenido

Introducción	1
Antecedentes	2
Contexto del sector gastronómico en Portoviejo	2
Composición empresarial y digitalización en Ecuador	2
Barreras para la digitalización de PYMES	3
Políticas nacionales de transformación digital	3
Justificación de una solución SaaS para PYMES gastronómicas	4
Definición del problema	4
Preguntas de investigación	7
Pregunta general	7
Preguntas específicas	7
Objetivos	8
Objetivo general	8
Objetivos específicos	8
Alcance	9
Método	12
Diseño de la Investigación	12

Enfoque y tipo	12
Población y Muestra	12
Población	12
Muestra y criterios de selección	13
Operacionalización de variables	13
Metodología	15
Técnicas e instrumentos de recolección de datos	17
Consideraciones éticas	19
Procedimiento	20
ETAPA 1: Planificación	20
Arquitectura de la Plataforma	23
Arquitectura de la base de datos	26
Roles, permisos y seguridad (JWT, RBAC)	29
ETAPA 2: Desarrollo	31
Desarrollo de la Plataforma SaaS	31
Desarrollo del frontend móvil (React Native + Expo)	32
Desarrollo del frontend panel web (Next.js + Tailwind CSS)	33
Desarrollo del backend (NestJS, capas, módulos)	34
ETAPA 3: Validación y Despliegue	41
Arquitectura del despliegue	41
Implementación y validación piloto	42
ETAPA 4: Resultados del software	44

Resultados	45
Introducción	45
Resultados del diagnóstico de digitalización	45
Resultados de la validación de usabilidad y adopción	45
Usabilidad del sistema	46
Adopción e impacto operativo	46
Costo estimado y financiamiento	46
Cronograma de actividades	47
Discusión	48
Objetivo General: La plataforma SaaS integrada y el significado de los hallazgos	48
Análisis de Objetivos Específicos y Contraste Bibliográfico	49
Diagnóstico de digitalización y brecha tecnológica (Objetivo Específico 1)	49
Arquitectura, diseño y desarrollo multiplataforma (Objetivos Específicos 2 y 3)	50
Módulos operativos y panel analítico (Objetivo Específico 4)	50
Chatbot RAG y atención al cliente (Objetivo Específico 5)	50
Usabilidad y aceptación del sistema (Objetivo Específico 6)	51
Limitaciones de la Investigación	51
Conclusiones	53
Referencias bibliográficas	55
Apéndice A	56

Lista de Figuras

1.	<i>Diagrama de flujo del funcionamiento general de la plataforma SaaS</i>	11
2.	Ciclo iterativo de la Investigación Basada en Diseño	16
3.	Arquitectura por capas de la plataforma SaaS	23
4.	Arquitectura de la aplicación móvil	24
5.	Arquitectura del panel web	25
6.	Arquitectura modular del backend (NestJS) planificada	25
7.	Diagrama entidad-relación (ER) del esquema de datos planificado	29
8.	Flujo de autenticación y autorización (JWT + RBAC)	31
9.	Flujo principal del frontend móvil para cliente y repartidor	33
10.	Flujo principal del panel web administrativo	35
11.	Flujo general del proceso de pedido	40
12.	Diagrama del módulo de inventario	40
13.	Diagrama del módulo de repartidores	41
14.	Arquitectura del despliegue de la plataforma	41
15.	<i>Cronograma de actividades del proyecto de titulación</i>	47
16.	<i>Diagrama entidad-relación de la base de datos de la plataforma SaaS</i>	57
17.	<i>Diagrama de red y arquitectura de la plataforma SaaS web y móvil</i>	57

Lista de Tablas

1.	<i>Operacionalización de variables e indicadores de medición</i>	14
2.	<i>Técnicas e instrumentos de recolección de datos</i>	18
3.	<i>Selección tecnológica para la arquitectura de la plataforma SaaS</i>	21
4.	<i>Tareas evaluadas en pruebas de usabilidad</i>	44
5.	<i>Costo estimado del proyecto</i>	46

Introducción

En las últimas décadas, la digitalización empresarial ha transformado radicalmente los modelos operativos y competitivos a nivel mundial. La adopción masiva de tecnologías digitales ha permitido a las organizaciones optimizar procesos, reducir costos operativos, mejorar la experiencia del cliente y tomar decisiones basadas en datos en tiempo real. Sectores como el comercio electrónico, la logística, los servicios financieros y la manufactura han experimentado cambios estructurales profundos mediante la implementación de sistemas ERP, plataformas de gestión integrada, automatización de procesos e inteligencia artificial aplicada. Sin embargo, esta transformación digital no ha sido uniforme ni equitativa: mientras que grandes corporaciones disponen de recursos técnicos, económicos y humanos para adoptar soluciones tecnológicas avanzadas, las Pequeñas y Medianas Empresas (PYMES) continúan enfrentando barreras significativas de acceso, costo, capacitación y adaptabilidad tecnológica.

A nivel latinoamericano y particularmente en Ecuador, las PYMES constituyen la columna vertebral del tejido empresarial, representando más del 90 % de las unidades productivas registradas y generando una proporción significativa del empleo nacional. No obstante, diversos estudios e informes institucionales evidencian que la mayoría de estas empresas mantiene niveles de digitalización limitados, especialmente en sectores tradicionales como la gastronomía, donde los procesos operativos continúan dependiendo de gestión manual, canales de comunicación no integrados y ausencia de herramientas automatizadas para la toma de decisiones. Esta brecha digital no solo limita su competitividad frente a empresas tecnificadas o plataformas digitales consolidadas, sino que también afecta su capacidad de adaptación, escalabilidad y sostenibilidad en mercados cada vez más exigentes, donde los consumidores demandan inmediatez, conveniencia, transparencia y experiencias omnicanal.

Frente a este panorama, las soluciones SaaS (Software as a Service) representan una oportunidad estratégica para democratizar el acceso a tecnología empresarial avanzada sin requerir inversiones iniciales elevadas, infraestructura propia o conocimientos técnicos especializados. Bajo esta perspectiva, el presente estudio se centra en evaluar la factibilidad, diseño, desarrollo y validación de una plataforma SaaS integral orientada específicamente a las PYMES gastronómicas de Portoviejo, una ciudad reconocida internacionalmente por la UNESCO como Ciudad Creativa de la Gastronomía, pero cuyos negocios gastronómicos locales aún enfrentan desafíos significativos en su proceso de modernización digital.

Antecedentes

Contexto del sector gastronómico en Portoviejo

Portoviejo fue declarada Ciudad Creativa de la Gastronomía por la UNESCO en 2019, un reconocimiento internacional que posiciona a la gastronomía local como un elemento estratégico tanto cultural como económico. Este distintivo subraya el potencial del sector para impulsar el desarrollo turístico, generar empleo y fortalecer la identidad regional. Las PYMES gastronómicas constituyen el núcleo de este sector, desempeñando un rol fundamental en la preservación de tradiciones culinarias y en la dinamización de la economía local.

Composición empresarial y digitalización en Ecuador

A nivel nacional, el tejido empresarial ecuatoriano está compuesto en gran parte por PYMES. Según datos del Observatorio de la PyME, cerca del 90,5 % de las empresas registradas en Ecuador son microempresas, seguidas por pequeñas y medianas. En el caso de la provincia de Manabí, donde se encuentra Portoviejo, las PYMES también juegan un rol importante en la generación de empleo y el desarrollo local.

Barreras para la digitalización de PYMES

Uno de los desafíos principales para estas empresas es la limitada adopción de tecnologías digitales. Si bien existe un reconocimiento creciente de su importancia, muchas PYMES siguen enfrentando barreras estructurales como costos elevados, falta de conocimientos técnicos, ausencia de capacitación especializada y sistemas tecnológicos poco adaptados a sus necesidades y capacidades operativas.

En muchos negocios gastronómicos pequeños, la gestión operativa (por ejemplo, inventarios y pedidos) continúa siendo manual o con herramientas básicas. Esta falta de automatización puede generar errores, desperdicios y dificultades para proyectar la demanda, lo que impacta negativamente en la rentabilidad y eficiencia. Además, el servicio al cliente todavía depende de canales tradicionales como llamadas telefónicas o WhatsApp, limitando la capacidad de ofrecer una experiencia profesional, rápida y escalable.

Por otra parte, la comisión que cobran algunas plataformas de delivery externas puede representar un costo significativo. Aunque no existen estudios exactos para todas las PYMES gastronómicas de Portoviejo, análisis de mercado muestran que estas comisiones pueden ser elevadas, reduciendo los márgenes de pequeños emprendimientos y afectando su sostenibilidad económica.

Políticas nacionales de transformación digital

La Agenda de Transformación Digital del Ecuador 2022–2025 establece una hoja de ruta institucional para impulsar la adopción de tecnologías en distintos sectores, incluyendo las PYMES. Entre sus objetivos está promover infraestructura tecnológica, facilitar la capacitación y crear un entorno favorable para que las empresas pequeñas accedan a soluciones digitales.

Justificación de una solución SaaS para PYMES gastronómicas

Frente a estas problemáticas, una solución SaaS (Software como Servicio) adaptada a PYMES gastronómicas locales emerge como una alternativa viable y estratégica. Un sistema SaaS puede ofrecer módulos integrados para la gestión de pedidos, control de inventarios, analítica empresarial y atención al cliente automatizada (mediante chatbots), sin requerir inversiones grandes en infraestructura o conocimientos técnicos avanzados por parte de los usuarios.

Por lo tanto, desarrollar una plataforma SaaS que responda a las necesidades operativas, técnicas y económicas de las PYMES gastronómicas de Portoviejo constituye una estrategia viable y alineada con las metas institucionales nacionales de digitalización y con las aspiraciones locales de modernización, competitividad y crecimiento sostenible del sector.

Definición del problema

En el sector gastronómico de la ciudad de Portoviejo, las Pequeñas y Medianas Empresas (PYMES) desempeñan un rol fundamental en la economía local, aportando significativamente a la generación de empleo, al dinamismo comercial y a la identidad gastronómica de la región. A pesar de su importancia, este sector enfrenta un proceso de digitalización lento e incompleto, caracterizado por la utilización de métodos tradicionales en la gestión operativa, administrativa y comercial. En un entorno donde la tecnología se ha convertido en un componente esencial para la competitividad y la sostenibilidad, muchas de estas empresas continúan utilizando procesos manuales para administrar pedidos, inventarios y servicios de delivery, dificultando su adaptación a las nuevas exigencias del mercado digital.

El problema principal identificado es la falta de soluciones tecnológicas accesibles, integradas y adaptadas al contexto local, que permitan a las PYMES gastronómicas gestionar de forma eficiente sus operaciones diarias. Actualmente, la mayoría de estos negocios no dispone de herramientas centralizadas para controlar inventarios, administrar pedidos, gestionar repartidores o brindar

atención al cliente mediante sistemas automatizados. Esto genera desorganización, tiempos de respuesta elevados, errores en la toma de pedidos y pérdidas económicas que afectan directamente la calidad del servicio y la rentabilidad del negocio. La ausencia de una plataforma tecnológica integral limita la capacidad de estas empresas para modernizarse y competir frente a cadenas más grandes o plataformas de delivery externas.

Diversos estudios y diagnósticos sobre transformación digital en Ecuador evidencian que las PYMES mantienen un nivel de adopción tecnológica limitado, especialmente en actividades tradicionales como la gastronomía. Aunque representan más del 90 % del tejido empresarial del país, informes como la Agenda de Transformación Digital del Ecuador 2022–2025 señalan que estas empresas aún enfrentan barreras significativas para modernizar sus procesos, debido a limitaciones económicas, escasa capacitación técnica y la falta de soluciones accesibles y adaptadas a su escala operativa. En Portoviejo, pese a ser reconocida por la UNESCO como Ciudad Creativa de la Gastronomía en 2019, la mayoría de los negocios gastronómicos continúa operando sin sistemas integrados de gestión para pedidos, inventarios y atención al cliente, lo cual genera procesos manuales, inconsistencias y baja eficiencia operativa. Esta evidencia muestra una brecha marcada entre el potencial gastronómico de la ciudad y el nivel real de digitalización presente en sus pequeñas y medianas empresas, lo que confirma la existencia de un problema estructural que afecta su competitividad y sostenibilidad.

Las causas del problema se relacionan principalmente con:

1. El alto costo de las soluciones tecnológicas disponibles en el mercado, que suelen estar orientadas a empresas medianas o grandes y no se ajustan al presupuesto de las PYMES locales.
2. La falta de capacitación tecnológica del personal, lo cual dificulta la adopción de plataformas complejas.
3. La ausencia de una oferta local de software adaptable, que incluya en una sola plataforma

módulos de delivery, inventarios, atención automatizada y analítica.

4. La dependencia de plataformas de terceros, que cobran comisiones elevadas y no permiten gestionar los procesos de manera personalizada.

Estas causas se combinan para generar un rezago tecnológico que limita el crecimiento y la modernización del sector gastronómico.

Si el problema no se soluciona, las PYMES gastronómicas de Portoviejo continuarán enfrentando dificultades operativas que derivan en pérdidas económicas, baja productividad y disminución de la calidad del servicio. La falta de digitalización provocará que estos negocios sigan siendo menos competitivos frente a empresas que ya adoptaron herramientas tecnológicas avanzadas. Además, se incrementarán los errores en la toma de pedidos, el desperdicio de insumos por falta de control de inventarios y los tiempos de entrega elevados debido a una mala gestión de repartidores. A largo plazo, esta situación limitará su capacidad para crecer, atraer nuevos clientes y mantenerse en un mercado que demanda rapidez, precisión y experiencia digital.

Resolver este problema es fundamental porque permitirá impulsar la transformación digital del sector gastronómico local, mejorar la competitividad de las PYMES y promover la adopción de tecnologías accesibles e inteligentes. El desarrollo de una plataforma SaaS integrada contribuirá a optimizar los procesos operativos, reducir costos, mejorar la toma de decisiones mediante analítica de datos y elevar la calidad del servicio al cliente mediante sistemas automatizados. Además, esta solución se alinea con los objetivos nacionales de digitalización y con la visión de Portoviejo como una ciudad innovadora y creativa. Abordar esta problemática beneficia no solo a los negocios, sino también al ecosistema económico local al fomentar la modernización, sostenibilidad y crecimiento del sector gastronómico.

Preguntas de investigación

Pregunta general

¿De qué manera el desarrollo e implementación de una plataforma SaaS integral puede contribuir a la digitalización, optimización operativa y mejora de la atención al cliente en las PYMES gastronómicas de la ciudad de Portoviejo?

Preguntas específicas

1. ¿Cuáles son las principales barreras tecnológicas, económicas y operativas que limitan la digitalización de las PYMES gastronómicas de Portoviejo?
2. ¿Qué módulos, características y funcionalidades son esenciales para mejorar la gestión de delivery, inventarios y procesos administrativos en el contexto gastronómico local?
3. ¿Cómo influye la integración de un módulo de inteligencia artificial basado en tecnología RAG en la eficiencia, rapidez y calidad de la atención al cliente?
4. ¿En qué medida la adopción de la plataforma SaaS desarrollada puede reducir costos operativos, optimizar tiempos de respuesta y mejorar la toma de decisiones estratégicas en las PYMES usuarias?
5. ¿Cuál es la percepción de usabilidad y satisfacción del usuario final respecto a la plataforma propuesta?

Objetivos

Objetivo general

Desarrollar y validar una plataforma SaaS que integre módulos de gestión operativa e inteligencia artificial, para optimizar los procesos de pedidos, inventarios, delivery y atención al cliente de las PYMES gastronómicas de Portoviejo.

Objetivos específicos

1. Realizar un diagnóstico del nivel de digitalización y un levantamiento detallado de requisitos funcionales y no funcionales con las PYMES objetivo.
2. Diseñar la arquitectura técnica, de software y de despliegue de la plataforma, definiendo su estructura modular y los protocolos de seguridad e integración.
3. Construir la plataforma mediante el desarrollo e integración simultánea de sus componentes modulares principales.
4. Implementar los módulos de gestión de pedidos (con geolocalización en tiempo real), control de inventarios, administración de repartidores y optimización de rutas, y el dashboard analítico con KPIs.
5. Integrar un chatbot basado en el modelo RAG dentro de la plataforma, para automatizar la atención al cliente y el soporte operativo.
6. Evaluar la plataforma integral mediante pruebas de usabilidad (SUS), rendimiento, funcionalidad y aceptación con usuarios reales en un entorno piloto, cuantificando su impacto en la eficiencia operativa.

Alcance

El presente proyecto se enmarca en el diseño, desarrollo y validación de una plataforma SaaS integral dirigida a la digitalización de las Pequeñas y Medianas Empresas (PYMES) del sector gastronómico de Portoviejo. Con el fin de establecer límites claros y garantizar una ejecución realista, se define el alcance del proyecto en dos dimensiones: los elementos que serán desarrollados e implementados, y aquellos que quedan excluidos de esta fase.

El proyecto abarca la creación completa de una solución tecnológica operativa, desde su concepción hasta su validación en un entorno controlado. En primer lugar, se realizará un análisis de requisitos basado en un diagnóstico del nivel de digitalización actual de las PYMES objetivo, mediante técnicas como encuestas y entrevistas, para derivar especificaciones funcionales y no funcionales. El diseño resultante contempla una arquitectura de software modular, compuesta por un backend desarrollado con NestJS que expone una API RESTful, una aplicación móvil multiplataforma construida con React Native para clientes y repartidores, y un panel administrativo web implementado con Next.js. La persistencia de datos se gestionará mediante una base de datos relacional PostgreSQL.

La plataforma integrará módulos funcionales centrales para la gestión operativa: un sistema de pedidos con registro de geolocalización de direcciones, un módulo de control de inventarios básico, un panel de administración de repartidores con procesos de aprobación y asignación manual de pedidos, y un dashboard analítico que presente indicadores clave de desempeño (KPIs) derivados de los datos transaccionales. Como componente innovador, se desarrollará e integrará un chatbot basado en el modelo de Generación Aumentada por Recuperación (RAG) para automatizar respuestas a consultas frecuentes de clientes.

La validación de la solución se llevará a cabo mediante un piloto controlado en al menos dos PYMES gastronómicas, donde se evaluará la usabilidad del sistema a través de la escala SUS (System Usability Scale), se verificará la funcionalidad de los flujos críticos y se medirá el impacto en métricas operativas como el tiempo de registro de pedidos y la tasa de errores. El despliegue para

esta validación se realizará en infraestructura en la nube, utilizando un servidor VPS para el backend y la base de datos, Vercel para el panel web, y se distribuirá la aplicación móvil como archivo APK para dispositivos Android.

Para garantizar la viabilidad y el enfoque del proyecto, se excluyen funcionalidades avanzadas y consideraciones de escala industrial. Esto incluye: el desarrollo de una arquitectura multi-tenant; la integración con pasarelas de pago; la optimización automática de rutas y el live-tracking de repartidores; las notificaciones en tiempo real (WebSockets/push); y módulos complementarios como marketing, reseñas o gestión de proveedores. Tampoco se contempla el desarrollo de una aplicación nativa para iOS, el despliegue de infraestructura de alta disponibilidad, una validación a escala sectorial, o un plan de soporte y mantenimiento post-implementación.

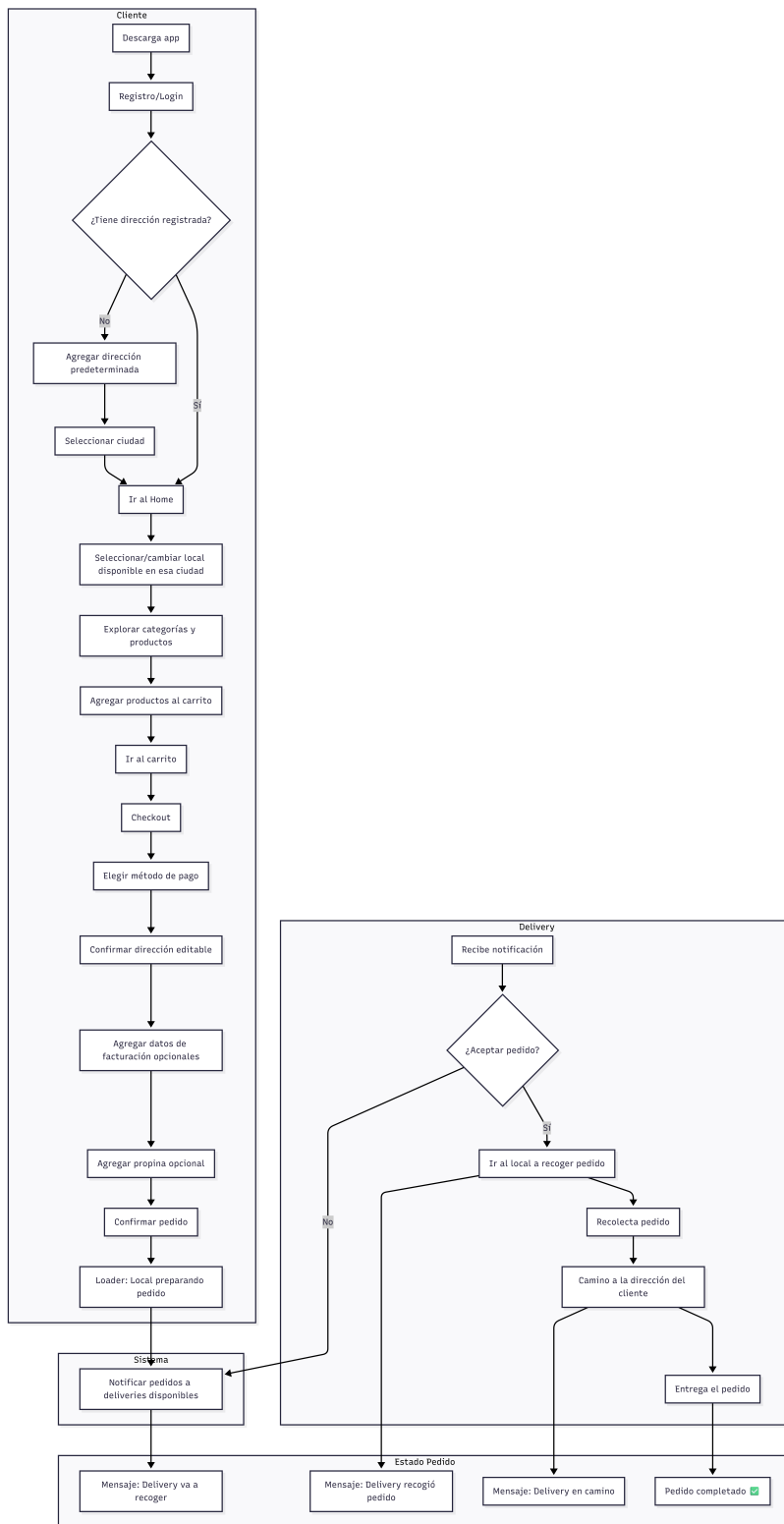


Figura 1. Diagrama de flujo del funcionamiento general de la plataforma SaaS

Método

Diseño de la Investigación

Enfoque y tipo

La investigación se desarrolló bajo un enfoque metodológico mixto (cualitativo-cuantitativo), con predominancia del componente aplicativo-desarrollativo. Se adoptó este enfoque para obtener una comprensión integral del problema: el componente cualitativo permitió profundizar en las percepciones, necesidades y experiencias de los usuarios, mientras que el cuantitativo posibilitó la medición objetiva de variables operativas y de usabilidad.

El estudio se clasifica como investigación aplicada de tipo tecnológica, dado que su propósito central es el diseño, construcción y validación de un artefacto de software (la plataforma SaaS) para resolver una problemática concreta en un contexto real. El diseño es no experimental, descriptivo y transversal, ya que se observaron y describieron las variables en su contexto natural sin manipulación deliberada, en un período específico.

Población y Muestra

Población

La población objetivo estuvo constituida por la totalidad de las Pequeñas y Medianas Empresas (PYMES) del sector gastronómico formalmente establecidas en la ciudad de Portoviejo, provincia de Manabí, Ecuador.

Muestra y criterios de selección

Se aplicó un muestreo no probabilístico por conveniencia y criterio. La muestra final estuvo conformada por dos PYMES gastronómicas. Los criterios de inclusión fueron:

- 1) Ser una PYME ubicada en Portoviejo.
- 2) Contar con servicio de consumo en el local y delivery.
- 3) Manifestar interés en mejorar procesos mediante digitalización.
- 4) Disponer de conectividad básica a internet.
- 5) Otorgar consentimiento para participar en todas las fases del estudio.

Se excluyeron establecimientos con operación irregular o que ya utilizaban sistemas de gestión integral.

Operacionalización de variables

Variable independiente: Plataforma SaaS. Se define como el artefacto tecnológico desarrollado, una plataforma de Software como Servicio que integra módulos de gestión operativa e inteligencia artificial. Su intervención consistió en su implementación y uso por parte de las PYMES piloto durante el período de validación.

Variables dependientes: Indicadores operativos. Son las métricas de desempeño que se buscó impactar con la implementación de la plataforma:

- 1) **Eficiencia en el registro de pedidos:** medida a través del tiempo promedio (minutos) para registrar un pedido y la tasa de errores de registro (porcentaje de pedidos con información incorrecta o incompleta).
- 2) **Eficacia del servicio de delivery:** medida mediante el tiempo promedio de entrega (minutos) desde la confirmación del pedido hasta su entrega.

- 3) **Productividad operativa:** medida a través del volumen diario de pedidos procesados.
- 4) **Calidad del servicio:** inferida a través de la tasa de cancelaciones de pedidos (porcentaje).
- 5) **Usabilidad del sistema:** medida a través del puntaje en la System Usability Scale (SUS), que evalúa la facilidad de uso percibida.

Tabla 1: Operacionalización de variables e indicadores de medición

Variable	dependiente	Definición operacional	Instrumento de medición	Umbral de éxito
Eficiencia en el registro de pedidos		Grado de optimización en la captura y procesamiento inicial de un pedido.	Ficha de observación estructurada (registro de tiempos y errores).	Reducción $\geq$ 25 % en el tiempo promedio de registro y en la tasa de errores.
Eficacia del servicio de delivery		Capacidad del sistema para gestionar y completar entregas dentro de un tiempo esperado.	Ficha de observación estructurada (registro de tiempos de entrega).	Reducción $\geq$ 15 % en el tiempo promedio de entrega.
Productividad operativa		Volumen de pedidos que el negocio puede procesar de manera efectiva en un día.	Registros administrativos generados por la plataforma (conteo de pedidos diarios).	Incremento $\geq$ 10 % en el número promedio de pedidos diarios procesados.
Calidad del servicio		Estabilidad y confiabilidad en la ejecución del servicio, minimizando fallas.	Registros administrativos de la plataforma (conteo de pedidos cancelados).	Reducción $\geq$ 20 % en la tasa de cancelaciones de pedidos.

Variable dependiente	Definición operacional	Instrumento de medición	Umbral de éxito
Usabilidad del sistema	Grado de facilidad, eficiencia y satisfacción con que los usuarios pueden emplear la plataforma.	Cuestionario estandarizado System Usability Scale (SUS).	Puntaje promedio ≥ 68 (umbral de aceptabilidad en la escala SUS).

Nota: Los umbrales de éxito para los indicadores operativos (reducción o incremento porcentual) se establecieron con base en expectativas realistas de mejora para un piloto de corto plazo, considerando la magnitud del cambio que justificaría la adopción de la herramienta. El umbral para el SUS se basa en la clasificación estándar de Bangor et al. (2009).

Metodología

El presente trabajo sigue la metodología Investigación Basada en Diseño (IBD), elegida por ser una metodología de investigación poderosa para desarrollar y evaluar artefactos (productos, procesos, métodos, modelos, etc.) que resuelven problemas del mundo real. No existe un único modelo canónico de fases, ya que distintos autores han propuesto esquemas similares pero con variaciones; sin embargo, todas comparten un núcleo cíclico e iterativo de construcción y evaluación. Se presenta el siguiente esquema integrando los modelos clave de Hevner et al. (2004) y Peffers et al. (2007).

Fase 1. El objetivo de esta fase fue comprender y definir claramente el problema. Se realizó una revisión de la literatura del tema, contextualizándolo en el ámbito global y local.

Fase 2. Una vez comprendido el problema, se establecieron los objetivos que busca conseguir el producto. En esta fase se definieron los requisitos funcionales y no funcionales de la plataforma.

Fase 3 y Fase 4. Forman el ciclo central de construcción-evaluación de la plataforma. Incluye desde

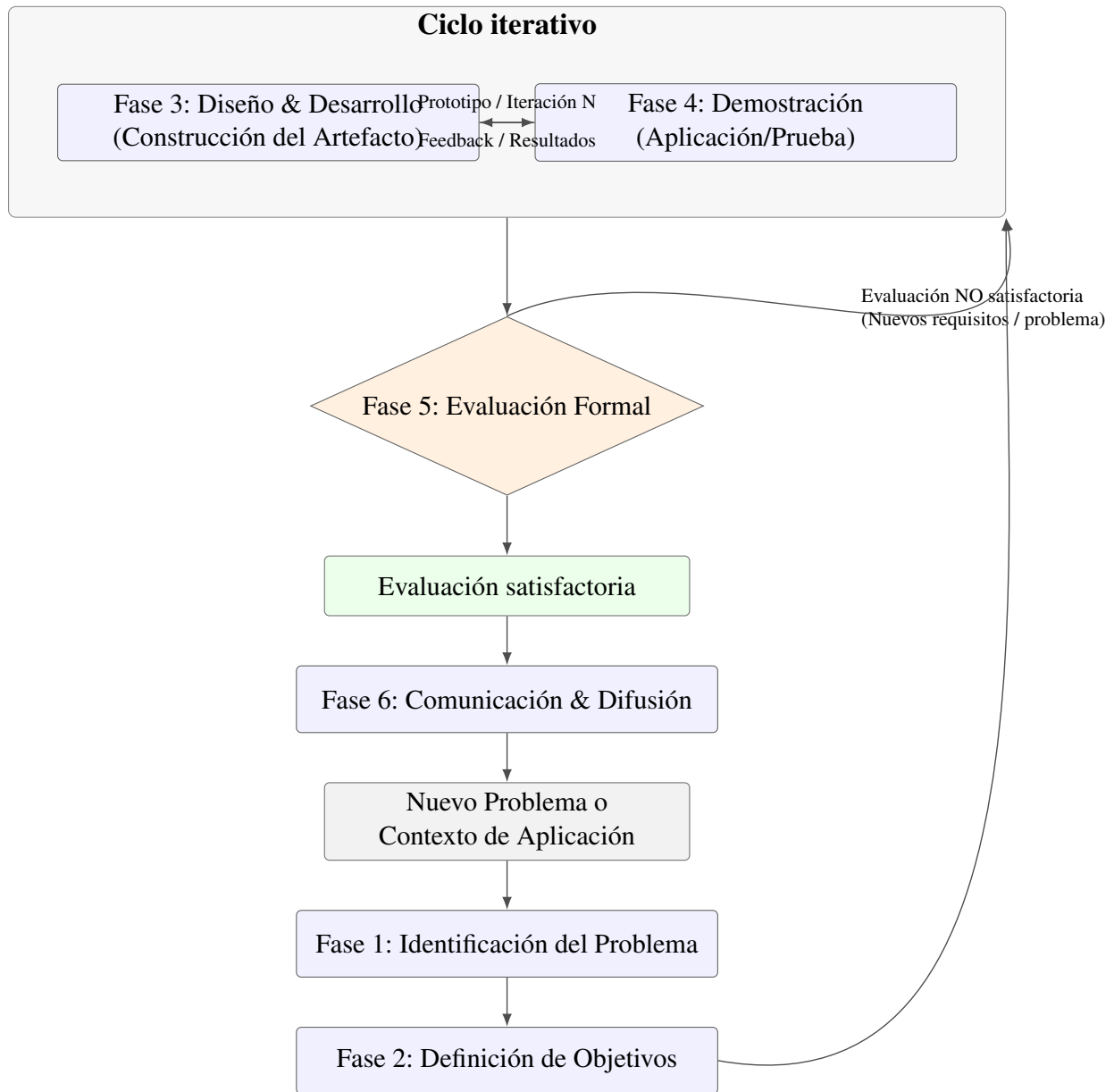


Figura 2. Ciclo iterativo de la Investigación Basada en Diseño

la selección de las herramientas tecnológicas, la arquitectura y el desarrollo de la solución. Se trabaja en ciclos cortos.

- Se diseña y desarrolla un incremento (un prototipo, una versión, un módulo).
- Inmediatamente se demuestra y prueba (en un entorno piloto, con usuarios, en simulación).
- Los resultados de esa demostración alimentan directamente el rediseño y desarrollo de la siguiente iteración.

Proceso ágil y sprints

La implementación ágil se estructuró en iteraciones cortas (sprints) con objetivos definidos: levantar requisitos mínimos viables, construir prototipos funcionales, realizar pruebas de usabilidad y ajustar funcionalidades a partir de la retroalimentación. Cada sprint concluyó con una revisión donde se consolidaron mejoras y se definieron tareas para el siguiente ciclo.

Fase 5. Comparar los resultados de la demostración con los objetivos definidos en la Fase 2. Se utilizaron métricas cuantitativas y cualitativas, las mismas que se detallan en la Tabla xx de la sección Técnicas e instrumentos de recolección de datos.

Fase 6. En esta fase se realiza la comunicación de los resultados, los mismos que se definen en la sección Resultados del presente Trabajo.

Técnicas e instrumentos de recolección de datos

En la siguiente tabla se muestran las técnicas e instrumentos utilizados, definiendo el momento de la aplicación y el público objetivo.

Tabla 2: *Técnicas e instrumentos de recolección de datos*

Técnica	Instrumento	Objetivo	Características	Momento de aplicación	Público objetivo
Entrevista	Guía de entrevista semiestructurada	Profundizar en los procesos operativos, necesidades específicas y expectativas para el levantamiento de requisitos.	Preguntas abiertas organizadas por áreas (pedidos, inventario, delivery). Grabada y transcrita.	Fase 1: Diagnóstico (línea base).	Propietarios de las PYMES.
Prueba de usabilidad	Cuestionario SUS (System Usability Scale)	Medir la usabilidad percibida (fidelidad de uso, satisfacción) de la plataforma implementada.	10 ítems con escala Likert de 5 puntos. Versión estandarizada en español. Confiabilidad ampliamente documentada.	Fase 6: Validación (post-intervención).	Usuarios finales de la plataforma (administradores, cajeros, repartidores).
Observación estructurada	Ficha de observación de indicadores operativos	Registrar datos cuantitativos objetivos sobre tiempos, errores y volúmenes de los procesos clave.	Diseñada para registrar: tiempos (min), conteo de errores, volumen de pedidos y cancelaciones.	En dos momentos: pre-intervención (línea base) y post-intervención (semana 3).	Procesos operativos de las PYMES piloto (Local A y B).

Técnica	Instrumento	Objetivo	Características	Momento de aplicación	Público objetivo
Revisión documental	Checklist de despliegue y registros administrativos	Verificar la correcta implementación técnica y recopilar datos productivos generados por el sistema.	Lista de verificación para servicios en la nube. Exploración de logs y reportes generados por la propia plataforma.	Fase 6: Validación (post-intervención).	Plataforma SaaS desplegada y sus registros automatizados.

Consideraciones éticas

Durante el desarrollo de la investigación se respetaron principios éticos fundamentales. La participación de las PYMES fue voluntaria y se garantizó la confidencialidad de la información proporcionada. Los datos recolectados se utilizaron exclusivamente con fines académicos, asegurando el anonimato de los participantes y evitando la divulgación de información sensible.

Se aplicó consentimiento informado, explicando objetivos del estudio, naturaleza de las actividades a realizar y derechos de los participantes (retirarse en cualquier momento, solicitar correcciones o eliminación de datos). La recolección y almacenamiento de información se efectuó siguiendo prácticas de minimización de datos, con acceso restringido y protección mediante credenciales. Los resultados se reportaron de forma agregada y sin identificadores, preservando la identidad de las PYMES. Asimismo, se contó con la autorización de los responsables de cada negocio para la aplicación de instrumentos y la evaluación del prototipo. Se evaluó el riesgo potencial de interferencia con operaciones cotidianas y se planificaron las actividades para minimizar impactos, realizando pruebas en momentos de baja carga cuando fue posible. Finalmente, se definieron periodos de retención y eliminación segura de datos conforme a buenas prácticas académicas y a la

normativa aplicable.

Procedimiento

Esta sección describe el diseño del prototipo de software y su implementación, correspondiente a las Fases 3, 4 y 5 de la metodología IBD. Se profundiza en el diseño arquitectónico, el desarrollo de los componentes y la implementación en entorno real.

Para una mejor comprensión metodológica, se detallan siguiendo la estructura:

- 1) Diseño arquitectónico.
- 2) Desarrollo.
- 3) Implementación y validación.

Este flujo garantizó una transición estructurada desde la conceptualización técnica hasta la evaluación del sistema en un entorno operativo real.

ETAPA 1: Planificación

Diseño de la arquitectura del sistema

Stack tecnológico. Esta etapa se dedicó a la definición de los cimientos tecnológicos y estructurales de la plataforma. A partir de los requisitos funcionales y no funcionales levantados, se realizó un análisis comparativo de tecnologías, frameworks y herramientas para cada capa del sistema. La selección final se basó en criterios de productividad del desarrollador, mantenibilidad a largo plazo, rendimiento, soporte comunitario y adecuación al contexto de las PYMES (recursos limitados, necesidad de soluciones ligeras). Las decisiones clave se resumen en la Tabla 2.

Tabla 3: *Selección tecnológica para la arquitectura de la plataforma SaaS*

Capa / Com-ponente	Tecnología seleccionada	Alternativas evaluadas	Criterio de selección principal
Backend (API y lógica)	NestJS (No-de.js/TypeScript)	Express.js (No-de.js), Django (Python), Spring Boot (Java)	Arquitectura modular por defecto, tipado estático (TypeScript) para reducir errores, inyección de dependencias integrada y ecosistema robusto para APIs escalables.
Base de datos	PostgreSQL	MySQL, MongoDB (NoSQL)	Modelo relacional adecuado para la integridad de datos transaccionales (pedidos, inventario). Balance entre rendimiento, características avanzadas y costo cero (open-source).
Aplicación móvil	React Native + Expo	Flutter (Dart), desarrollo nativo (Kotlin/Swift)	Multiplataforma real (iOS/Android desde un solo código), Expo para agilizar ciclo de desarrollo (builds, testing) y acceso a APIs nativas sin complejidad.

Capa / Componente	Tecnología seleccionada	Alternativas evaluadas	Criterio de selección principal
Panel web admin	Next.js (React) + Tailwind CSS	React (Create React App), Vue.js, Angular	Renderizado híbrido (SSR/CSR) de Next.js para mejor SEO y performance de carga; Tailwind CSS para estilizado ágil y consistente.
Autenticación	JWT (JSON Web Tokens) + bcrypt	Sesiones en servidor, OAuth 2.0 (third-party)	Stateless y escalable para APIs REST. Simplicidad de implementación y seguridad suficiente para el alcance del piloto.
Contenedorización	Docker	—	Estandarización del entorno de desarrollo y despliegue, garantizando consistencia entre equipos y facilitando el deployment en la nube.
Despliegue backend/DB	VPS (Hetzner)	AWS/Azure (cloud), Railway, Render	Costo-eficiencia y control total del servidor para un piloto, con rendimiento predecible y facturación simple.

Capa / Componente	Tecnología seleccionada	Alternativas evaluadas	Criterio de selección principal
Despliegue panel web	Vercel	Netlify, GitHub Pages	Integración nativa con Next.js, despliegue automático por Git, CDN global y SSL gratuito, minimizando tareas de DevOps.

Arquitectura de la Plataforma

Una vez establecidas las herramientas tecnológicas, se definió y estructuró una arquitectura por capas:

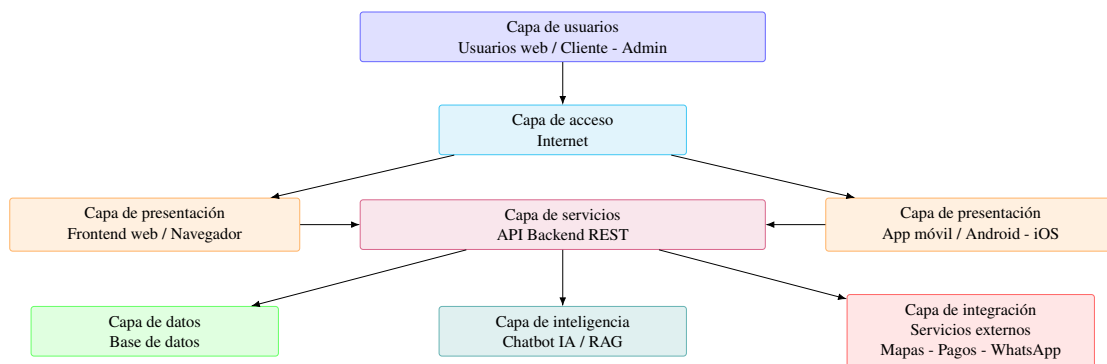


Figura 3. Arquitectura por capas de la plataforma SaaS

Los componentes principales de la plataforma son:

- Aplicación móvil para clientes y repartidores.
- Aplicación web para el panel de administración.
- Servicios backends con sus respectivas APIs.

Para tener un mejor detalle de cada componente, se presenta a continuación la arquitectura de la aplicación móvil, la arquitectura de la aplicación web y el diagrama de la base de datos.

Arquitectura de la aplicación móvil

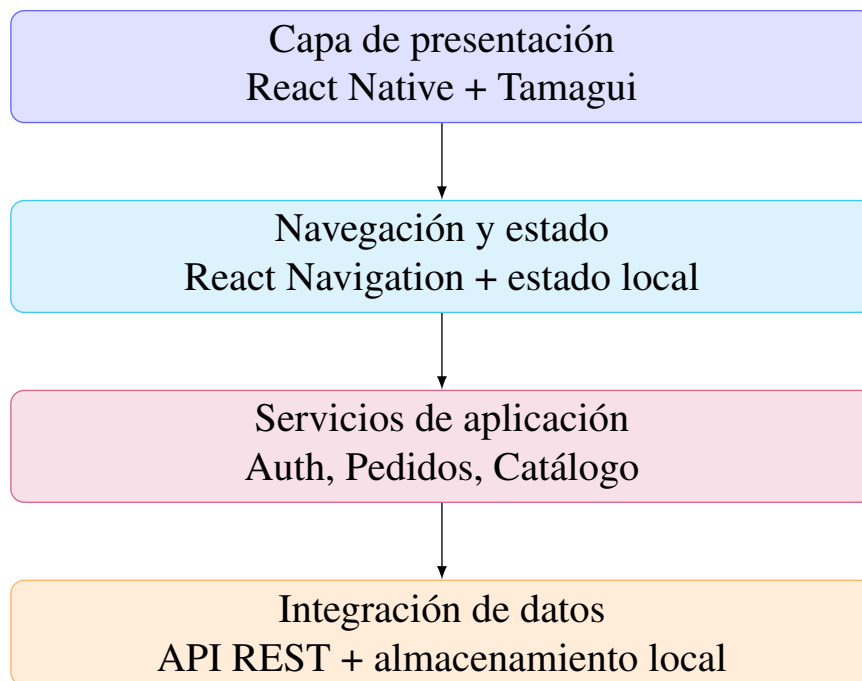


Figura 4. Arquitectura de la aplicación móvil

Arquitectura del panel web

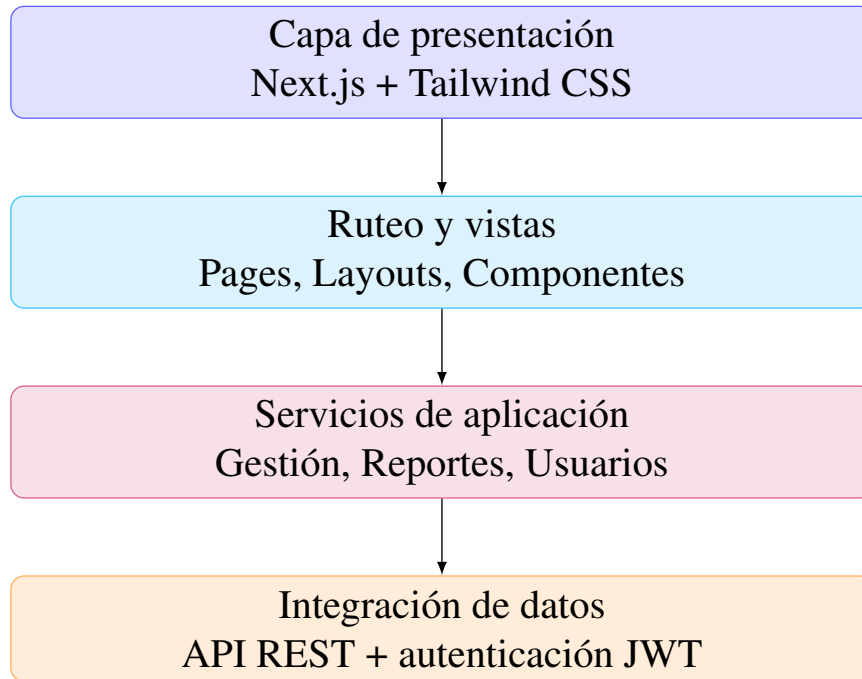


Figura 5. Arquitectura del panel web

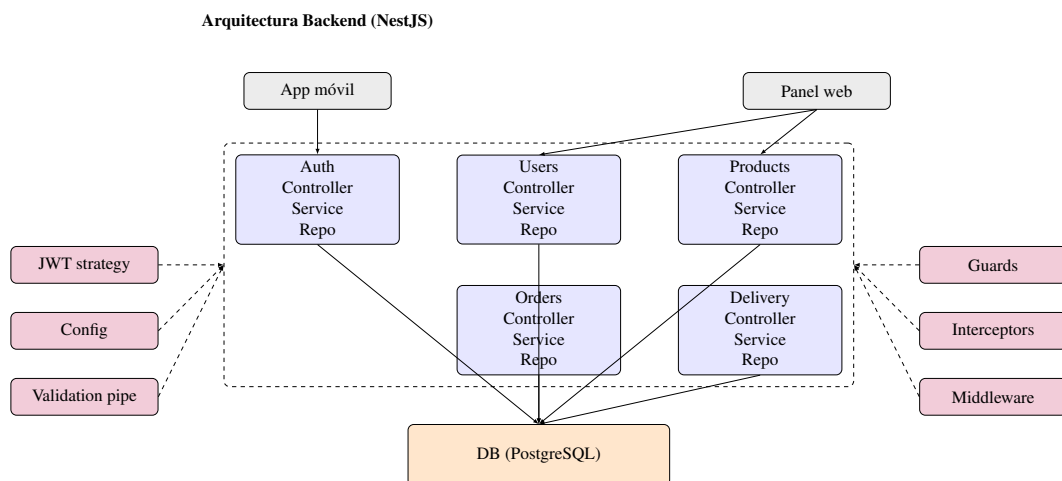


Figura 6. Arquitectura modular del backend (NestJS) planificada

Arquitectura de la base de datos

Objetivo del modelo de datos: garantizar integridad referencial, consultas eficientes y soporte a multi negocio con segregación lógica por empresa.

Para esta plataforma se optó por un modelo de base de datos relacional, adecuado para representar las entidades y relaciones propias del dominio (usuarios, productos, pedidos, etc.) garantizando consistencia y permitiendo consultas complejas (p. ej., obtener todos los pedidos de un cliente filtrados por fecha, o los productos más vendidos por categoría). En particular, usamos un motor SQL (como PostgreSQL) con un esquema relacional tradicional.

La decisión de usar un RDBMS sobre una base NoSQL estuvo motivada por la necesidad de integridad referencial (por ejemplo, asegurar que un pedido refiere a un usuario existente, o que un producto pertenece a una categoría válida) y la facilidad para realizar uniones (JOINS) entre tablas para extraer informes o datos combinados, algo muy útil de cara a las futuras métricas y reportes.

Modelo Entidad-Relación principal: A grandes rasgos, se definen las siguientes entidades (tablas) en el esquema de datos.

Usuario: Representa a los usuarios de la plataforma. Incluye id (clave primaria), nombre, email, contraseña encriptada, rol (cliente, repartidor, administrador) y, en el caso de repartidores, un estado de aprobación. El rol se usa para aplicar lógicas diferenciadas; el JWT del usuario incorpora su rol para decisiones de autorización.

Producto: Ítem disponible para pedido con campos como id, nombre, descripción, precio e imagen. Cada producto se asocia a una categoría y, en arquitectura multinegocio, al negocio propietario.

Categoría: Agrupa productos. Contiene id y nombre, pudiendo admitir jerarquías. En enfoque multinegocio, las categorías se vinculan al negocio correspondiente para evitar conflictos y permitir catálogos por empresa.

Pedido: Registra cada orden realizada por un cliente con id, fecha/hora, estado (pendiente, asignado, en camino, entregado, cancelado, etc.), referencia al cliente y, cuando aplica, al repartidor

asignado. El detalle del pedido se modela con una tabla de items (OrderItem) que normaliza la relación muchos a muchos con Producto.

Negocio: Representa cada empresa dentro de la plataforma (tenant). Incluye id, nombre, branding (logo, colores), contacto y configuraciones. Otras entidades se relacionan con Negocio para segmentar datos por empresa.

Usuarios, Productos, Categorías y Pedidos: en enfoque multinegocio, estas entidades incluyen una referencia al negocio propietario para filtrar datos por empresa y mantener segregación lógica.

Entidades de soporte: según el alcance, pueden añadirse Pago, Direcciones o Notificaciones.

Además de las relaciones, se definieron índices en campos de búsqueda frecuente (por ejemplo, estado y fecha en pedidos, nombre en productos) para optimizar consultas, y restricciones de unicidad en claves naturales donde corresponde (emails de usuarios, nombres de categorías por negocio). Se cuidó la normalización para evitar duplicidades innecesarias, manteniendo un equilibrio pragmático entre integridad y rendimiento. En entidades con alto volumen de escritura (pedidos y sus items) se contempló el uso de transacciones para asegurar consistencia en operaciones compuestas.

En la implementación con TypeORM, cada una de estas entidades se refleja en una clase con decoradores @Entity, y las relaciones se anotan con @ManyToOne, @OneToMany, etc., para que el ORM tienda las foreign keys. Por ejemplo, la entidad Pedido tendría @ManyToOne(() => User, user => user.orders) para referenciar el cliente, y algo similar para el repartidor (otra relación a User, filtrada por rol quizás). También un @ManyToOne(() => Business, ...) si soportamos multi-negocio en pedidos.

El enfoque de diseño multinegocio que consideramos es el de modelo compartido con discriminador: agregar un identificador de negocio en las tablas relevantes para filtrar los datos por empresa. Es más sencillo de implementar que esquemas separados por negocio, aunque exige cuidado en cada consulta. Al incluir businessId en Productos, Categorías y Pedidos, todas las consultas deben filtrar por el negocio correspondiente para evitar mezclar datos de distintas

empresas (Scalzotto, s.f.).

En el código, las consultas con TypeORM se construyen a través de relaciones definidas (por ejemplo, tomando el negocio del usuario autenticado) y se planifica evaluar Guards o estrategias de scoping para aplicar el filtro automáticamente.

Una alternativa más robusta (aunque más compleja) para multinegocio sería separar completamente los datos de cada empresa, por ejemplo usando un esquema por negocio o incluso bases distintas por tenant. NestJS/TypeORM soportan multi-tenancy mediante múltiples conexiones o esquemas, pero dado el alcance inicial se optó por un único esquema con campo discriminador, manteniendo la posibilidad de migrar a otro enfoque si se requiere mayor aislamiento.

Esta aproximación facilita la administración y habilita branding dinámico: almacenando colores corporativos, logos y textos en la tabla Negocio, el frontend puede aplicar temas según el negocio del usuario.

Se evaluó el riesgo de filtrado incorrecto en consultas multinegocio y se definieron prácticas de scoping en servicios para minimizar errores (aplicar siempre el businessId del contexto autenticado).

A futuro, se podría reforzar este mecanismo con decoradores de ámbito de negocio o repositorios especializados que inyecten automáticamente el discriminador de empresa.

En cuanto a la integridad referencial se han definido claves foráneas para asegurar la consistencia: por ejemplo, si se elimina o desactiva un negocio, podríamos optar por también desactivar cascada sus productos y categorías asociados (o prevenir la eliminación si tiene pedidos activos, dependiendo de reglas de negocio). Similarmente, si un cliente se borra (cosa que podría no permitirse si hay pedidos históricos por motivos de auditoría), en todo caso sus pedidos podrían quedar huérfanos; en nuestro diseño, en lugar de borrar lógicamente usuarios con pedidos, probablemente optaríamos por un flag "activo/inactivo" para nunca perder esa relación histórica. Todas estas decisiones de integridad se configuran bien en un modelo relacional mediante ON DELETE/UPDATE constraints o manejadas a nivel de servicio en NestJS antes de realizar

operaciones de borrado.

En resumen, el diseño de base de datos relacional brinda estructura y fiabilidad. El modelo multi negocio mediante identificador de empresa en cada entidad principal equilibra simplicidad y extensibilidad: aunque la app actual opera con un único negocio, se sientan las bases para diferenciar datos por cliente empresarial en el futuro. Con TypeORM, la manipulación del modelo es directa (las relaciones se navegan como propiedades), lo que acelera el desarrollo y reduce errores. Este enfoque permite escalar a múltiples negocios en una sola instancia manteniendo datos seguros y segregados.

Finalmente, se prevé definir estrategias de respaldo y recuperación ante desastres acordes al tamaño del despliegue, así como mecanismos de migración segura para cambios de esquema futuros (migraciones versionadas, ventanas de mantenimiento y validación posterior a la actualización).

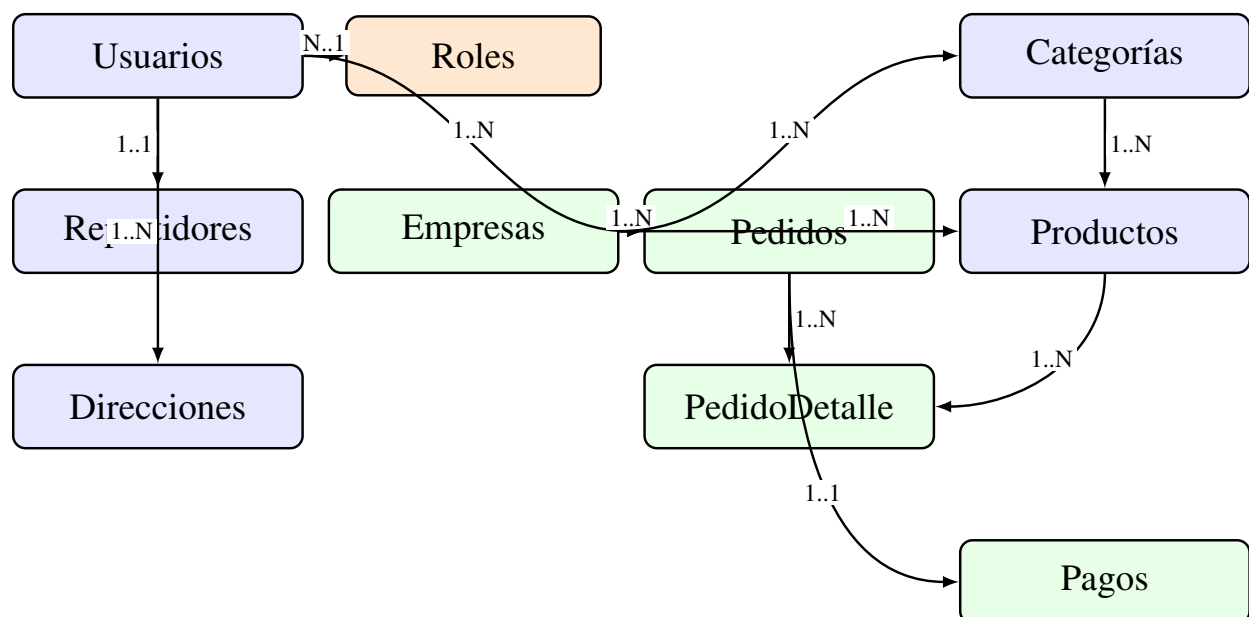


Figura 7. Diagrama entidad-relación (ER) del esquema de datos planificado

Roles, permisos y seguridad (JWT, RBAC)

La plataforma implementa un esquema de roles y permisos (RBAC) para garantizar que cada usuario solo realice las acciones que le corresponden (Logto, s.f.). RBAC gestiona "quién puede

hacer qué" agrupando permisos en roles; a cada usuario se le asigna uno o varios roles y cada rol confiere permisos sobre funciones, APIs o datos.

Administrador (ADMIN): Usuario del panel web con privilegios completos. Puede crear y editar productos y categorías, ver todos los pedidos, gestionar repartidores (aprobar/rechazar) y configurar la plataforma. En entornos multinegocio podría existir un administrador por empresa.

Cliente (CLIENTE): Usuario de la app móvil que realiza pedidos. Puede registrarse o iniciar sesión, ver y buscar productos, crear pedidos, consultar su estado y editar su perfil.

Repartidor (DELIVERY): Usuario de la app móvil que toma y entrega pedidos. Puede ver pedidos pendientes, aceptar/declinar encargos, marcar entregas y revisar su historial. Requiere aprobación administrativa antes de operar.

Estos roles se asignan en la base de datos, ya sea mediante un campo role en la tabla de usuarios o mediante una relación muchos a muchos Usuario-Rol (si quisiéramos soportar múltiples roles por usuario). En este proyecto, dado que los roles son bien diferenciados, optamos por un campo simple enumerado ('ADMIN' | 'CLIENTE' | 'DELIVERY') en la entidad Usuario para la simplicidad.

La autorización por roles se implementa de forma declarativa en endpoints críticos, con mensajes de error consistentes y registro de intentos no autorizados para auditoría. En cuanto a autenticación, se definieron políticas de expiración de tokens equilibradas entre seguridad y usabilidad; por simplicidad, inicialmente no se emplean tokens de actualización (refresh), aunque se prevé su inclusión si el uso lo amerita. Se recomienda almacenar credenciales y tokens de forma segura en cliente (cookies HttpOnly en web, almacenamiento seguro en móvil) y evitar exposición en logs.

Para la autenticación y autorización, utilizamos JWT (JSON Web Tokens) como mecanismo de autenticación stateless. JWT es un estándar abierto (RFC 7519) ampliamente utilizado para transmitir información de forma segura en forma de token JSON firmado. De hecho, la autenticación con tokens JWT se ha convertido en un estándar de facto para proteger los endpoints sensibles de aplicaciones web y APIs (Aguilera,., xm XX implementamos JWT mediante la integración de Passport.js en NestJS con la estrategia passport-jwt.

El flujo de autenticación y autorización se muestra en el siguiente diagrama, y el detalle del mismo se encuentra en el Anexo XX.

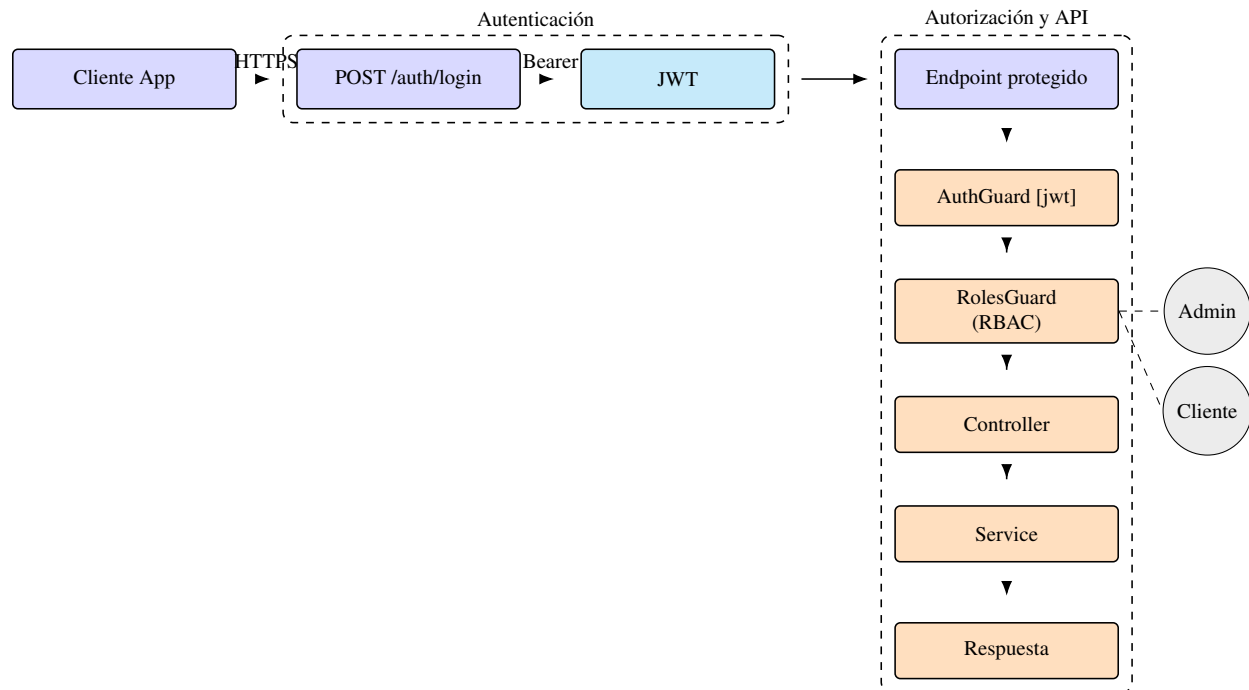


Figura 8. Flujo de autenticación y autorización (JWT + RBAC)

ETAPA 2: Desarrollo

Desarrollo de la Plataforma SaaS

Los Objetivos Específicos 3, 4 y 5 se centraron en la construcción iterativa e integración de todos los componentes del sistema. Se siguió una metodología ágil, organizando el trabajo en sprints de dos semanas (Anexo 5). El desarrollo fue modular y paralelo. Configuración inicial: se establecieron los tres repositorios (backend, móvil, web), se configuraron las herramientas de contenedorización (Docker) y se definió la estructura base de la base de datos en PostgreSQL.

Desarrollo del frontend móvil (React Native + Expo)

Objetivo del frontend: ofrecer una experiencia clara y eficiente para clientes y repartidores, con navegación predecible, validaciones tempranas y comunicación robusta con la API. El frontend móvil se implementó con React Native y Expo, lo que permite desarrollar una aplicación nativa multiplataforma a partir de una sola base de código.

React Native facilita la reutilización entre iOS y Android sin comprometer el rendimiento; Expo simplifica el ciclo de vida de la aplicación al ofrecer herramientas integradas para compilar, desplegar y probar sin configurar proyectos nativos desde cero (campusMVP, s.f.). Además, Expo expone funcionalidades nativas desde JavaScript (cámara, notificaciones, sensores) y habilita un flujo ágil con recarga en vivo y publicación mediante enlaces o códigos QR, favoreciendo la colaboración y las pruebas (campusMVP, s.f.).

La aplicación ofrece flujos diferenciados para dos roles: Cliente y Repartidor, además de pantallas de inicio de sesión y registro. La navegación se gestiona con la librería de React Native (p. ej., React Navigation), que permite transicionar entre vistas, mantener historiales y pasar parámetros. Cada rol visualiza únicamente las opciones pertinentes; esta segregación mejora la experiencia y refuerza la seguridad, complementada con validaciones en el backend.

La arquitectura del frontend móvil privilegia la simplicidad y la previsibilidad del estado. Se emplea estado local y patrones de elevación de estado donde corresponde, con manejo de formularios controlados y validación básica en cliente (tipos de entrada, obligatoriedad, formatos). La comunicación con la API se realiza mediante solicitudes HTTP que envían y reciben JSON; se contemplan casos de error (credenciales inválidas, falta de conectividad, respuestas 4xx/5xx) con mensajes claros al usuario y estrategias de reintento cuando es pertinente. La interfaz se diseñó con principios de consistencia visual y accesibilidad, cuidando contrastes, tamaños de toque y retroalimentación tras acciones. Para el diseño visual se empleó Tamagui, biblioteca de UI y sistema de estilos compatible con React Native y React web, que provee componentes y temas para construir interfaces coherentes y responsivas (Tamagui, s.f.). En el proyecto se estilizaron botones,

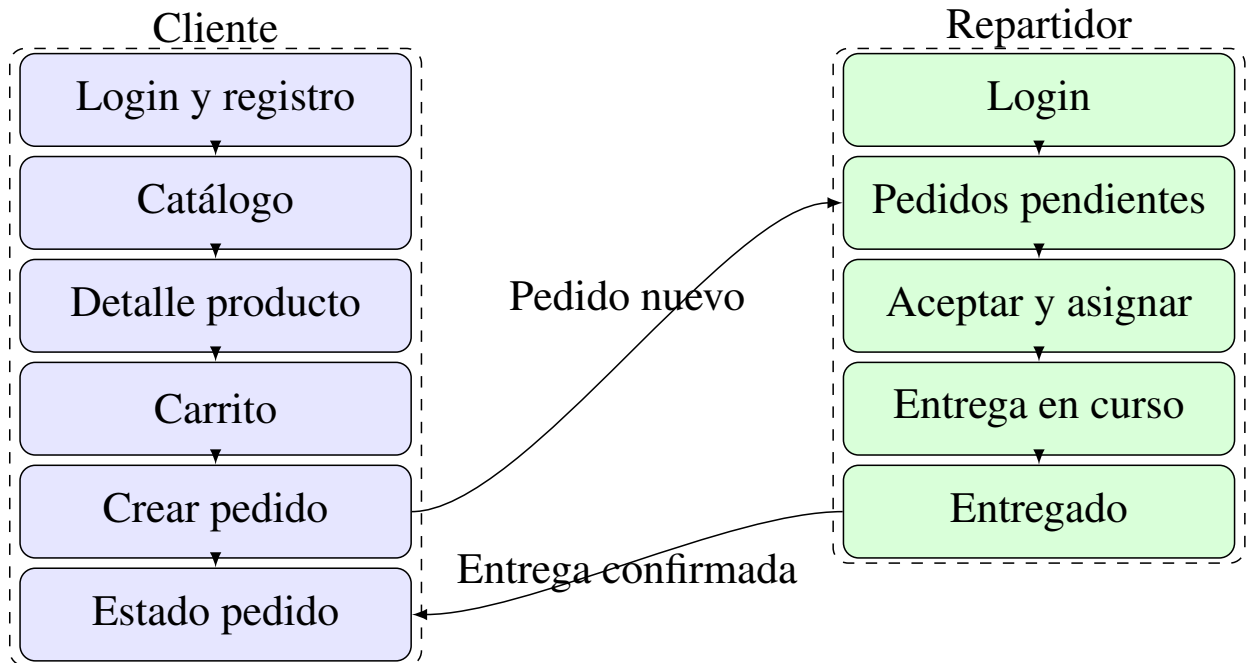


Figura 9. Flujo principal del frontend móvil para cliente y repartidor

tarjetas y formularios con componentes Tamagui, logran do consistencia y mantenibilidad. Su enfoque multiplataforma facilita la reutilización futura de componentes en una versión web, así como la personalización temática por negocio.

Desarrollo del frontend panel web (Next.js + Tailwind CSS)

Objetivo del panel web: proveer herramientas administrativas para gestionar catálogo, pedidos y repartidores, con formularios validados, vistas responsivas y flujos CRUD claros.

Se desarrolló además un panel web administrativo con React y Next.js. Aunque el panel es interno, se aprovechó el renderizado del lado del servidor (SSR) en vistas específicas de dashboard cuando resultó conveniente.

El estilado se realizó con Tailwind CSS, framework utility-first que permite componer diseños rápidos y consistentes mediante clases utilitarias (Tailwind CSS, s.f.). Su configuración admite alta personalización, útil para adaptar paletas o temas por cliente.

En el panel se aplicaron formularios validados con reglas de negocio simples (campos requeridos,

formatos, rangos) y se implementaron flujos habituales de CRUD con confirmaciones y mensajes de éxito/error.

La estructura de navegación organiza secciones por dominio (productos, categorías, pedidos, repartidores) y proporciona filtros básicos (por estado, fecha) para facilitar la gestión. Se cuidó la responsividad para operar en distintos tamaños de pantalla, y se diseñaron tablas y listas con paginación prevista para escenarios de datos voluminosos.

El panel permite crear y editar productos, gestionar categorías y aprobar o rechazar repartidores que solicitan unirse. Los formularios incorporan validaciones de campos; las vistas de lista presentan pedidos en curso y solicitudes pendientes.

Cuando un repartidor se registra desde la app móvil, su cuenta queda "pendiente" hasta la revisión administrativa, tras la cual puede ser habilitada o rechazada, garantizando control de calidad.

Ambos frontends consumen el mismo backend mediante API REST y JSON. La separación de responsabilidades mantiene el frontend enfocado en la experiencia y presentación, mientras la lógica de negocio y la persistencia residen en el backend.

En materia de rendimiento y experiencia, se optimizaron listas y vistas frecuentes y se cuidó la retroalimentación visual en operaciones que pueden tardar (carga de catálogo, envío de pedidos). Se delinearon buenas prácticas para el manejo de imágenes (compresión, tamaños adecuados) y para el control de estados vacíos y errores de red, mejorando la robustez de la interfaz en condiciones reales.

Desarrollo del backend (NestJS, capas, módulos)

Objetivo del backend: exponer servicios estables y seguros, aplicar reglas de negocio y coordinar persistencia con una arquitectura modular mantenible.

El backend se implementó con NestJS, framework de Node.js de arquitectura modular que promueve buenas prácticas mediante el uso de TypeScript, decoradores e inyección de

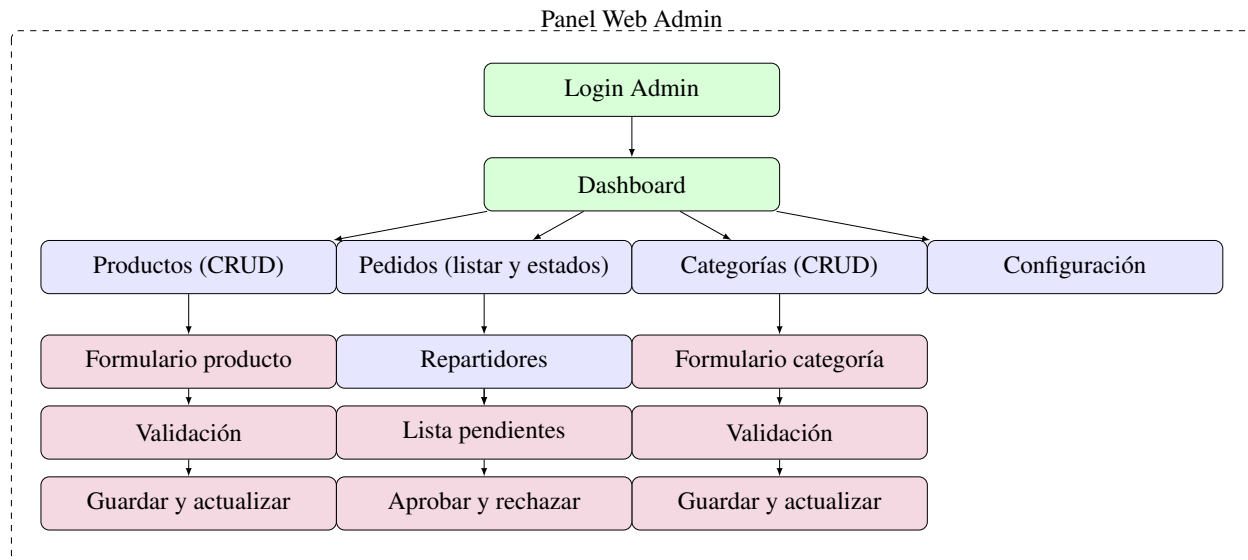


Figura 10. Flujo principal del panel web administrativo

dependencias. Frente a Express puro, NestJS aporta una estructura clara basada en módulos, controladores y servicios, facilitando la escalabilidad y el mantenimiento del código por dominios de negocio (Usuarios, Productos, Pedidos, Autenticación, entre otros).

Cada módulo agrupa tres componentes fundamentales: un controlador que define endpoints y atiende solicitudes HTTP; un servicio que concentra la lógica de negocio; y una capa de datos o repositorio que interactúa con la base de datos, ya sea mediante un ORM o consultas específicas (Dragos, s.f.). Siguiendo este patrón, el módulo de Autenticación expone rutas para login y registro en su controlador; el servicio valida credenciales y emite tokens JWT; y se apoya en el módulo de Usuarios para verificar identidad. De modo análogo, el módulo de Productos ofrece rutas para listar y crear productos (acción reservada al administrador), mientras su servicio aplica reglas de negocio y sus entidades mapean a tablas mediante el ORM.

La capa de control incluye manejo de errores y respuestas consistentes; se emplean filtros de excepciones para capturar y formatear fallos (validación, autorización, acceso a datos) en códigos HTTP adecuados. Los servicios encapsulan reglas de negocio con claridad, evitando dependencias con detalles de transporte HTTP. La configuración de la aplicación se centraliza con lectura de variables de entorno, diferenciando perfiles de ejecución (desarrollo, pruebas, producción) y

resguardando credenciales. El uso integral de TypeScript habilita detección temprana de errores y mejor navegación del código.

Se definieron DTOs para tipar y validar datos de entrada y salida; la integración con `class-validator` y `class-transformer`, junto con pipes de validación global, permite rechazar automáticamente solicitudes mal formadas y entregar al controlador datos ya validados. La inyección de dependencias mantiene el código desacoplado y testeable. Marcando clases como `@Injectable()`, estas pueden solicitarse en constructores de otras clases y Nest provee instancias adecuadas. Por ejemplo, `OrdersService` puede inyectar el servicio de notificaciones o el repositorio de pedidos sin instanciarlos manualmente, siguiendo el principio de inversión de dependencias.

Las responsabilidades se mantienen unidireccionales: el controlador recibe la petición y delega; el servicio ejecuta reglas de negocio y coordina repositorios; y la capa de datos interactúa exclusivamente con la base (Dragos, s.f.; Q2B Studio, s.f.). Esta separación, alineada con Clean Architecture y DDD, evita mezclas de responsabilidades y facilita el reemplazo de componentes sin afectar capas superiores. Esta arquitectura favorece la mantenibilidad y la escalabilidad. Cambios en la forma de acceder a datos (p. ej., migrar de TypeORM a otro ORM o a microservicios) impactan principalmente en la capa de datos.

Asimismo, simplifica las pruebas unitarias al permitir testear servicios o controladores en aislamiento mediante simulaciones. La persistencia se maneja con TypeORM, integrado mediante `@nestjs/typeorm`. Las entidades TypeScript decoradas representan tablas; los repositorios y el QueryBuilder facilitan operaciones sin escribir SQL manual. TypeORM soporta múltiples motores (PostgreSQL, MySQL, etc.) y ofrece migraciones de esquema; aunque inicialmente se usó sincronización automática, se prevé incorporar migraciones controladas.

La organización por módulos comprende, entre otros: Users (usuarios y roles, integrado con Auth), Products y Categories (gestión de catálogo), Orders (creación, asignación y estados de pedidos), Delivery (operativas de repartidores, según el diseño), y Auth (autenticación JWT y guardias de acceso). NestJS se integra con Passport.js para la autenticación; se definió una estrategia JWT

personalizada (detallada en la sección de seguridad).

Además, se emplearon Guards, Interceptors y Middleware en puntos específicos: un middleware global de registro para depurar peticiones, guards para control de acceso (AuthGuard [jwt] y un guard de roles) e interceptores para formatear respuestas y medir rendimiento. Se prevé incorporar un interceptor global que mida el tiempo de ejecución y el conteo de accesos.

En validación de datos, se activó un pipe global para que los DTOs sean transformados y verificados antes de llegar a los controladores, lo que robusteció la API ante entradas malformadas. Se consideran medidas adicionales de endurecimiento para producción, como cabeceras de seguridad, CORS configurado por origen y limitación de tasa de peticiones. 24 En cuanto al despliegue, el backend está preparado para ser ejecutado como una aplicación Node independiente. Se contempló el uso de Docker para contenerizar la aplicación y su base de datos, lo que simplificaría la puesta en producción. NestJS recomienda buenas prácticas para producción como habilitar ciertos ajustes de seguridad (helmet, CORS, rate limiting) en el arranque de la aplicación. Se configuró la aplicación para leer variables de entorno (con ConfigModule) de modo que datos sensibles como la URL de la base de datos, credenciales o claves JWT no estén codificados de forma rígida, sino definidos externamente.

(debe ser anexo de la base de datos) Usuario: Representa a los usuarios de la plataforma. Incluye id (clave primaria), nombre, email, contraseña encriptada, rol (cliente, repartidor, administrador) y, en el caso de repartidores, un estado de aprobación. El rol se usa para aplicar lógicas diferenciadas; el JWT del usuario incorpora su rol para decisiones de autorización.

Producto: Ítem disponible para pedido con campos como id, nombre, descripción, precio e imagen. Cada producto se asocia a una categoría y, en arquitectura multinegocio, al negocio propietario.

Categoría: Agrupa productos. Contiene id y nombre, pudiendo admitir jerarquías. En enfoque multinegocio, las categorías se vinculan al negocio correspondiente para evitar conflictos y permitir catálogos por empresa.

Pedido: Registra cada orden realizada por un cliente con id, fecha/hora, estado (pendiente,

asignado, en camino, entregado, cancelado, etc.), referencia al cliente y, cuando aplica, al repartidor asignado. El detalle del pedido se modela con una tabla de items (OrderItem) que normaliza la relación muchos a muchos con Producto.

Negocio: Representa cada empresa dentro de la plataforma (tenant). Incluye id, nombre, branding (logo, colores), contacto y configuraciones. Otras entidades se relacionan con Negocio para segmentar datos por empresa.

Usuarios, Productos, Categorías y Pedidos: en enfoque multinegocio, estas entidades incluyen una referencia al negocio propietario para filtrar datos por empresa y mantener segregación lógica.

Entidades de soporte: según el alcance, pueden añadirse Pago, Direcciones o Notificaciones.

Además de las relaciones, se definieron índices en campos de búsqueda frecuente (por ejemplo, estado y fecha en pedidos, nombre en productos) para optimizar consultas, y restricciones de unicidad en claves naturales donde corresponde (emails de usuarios, nombres de categorías por negocio). Se cuidó la normalización para evitar duplicidades innecesarias, manteniendo un equilibrio pragmático entre integridad y rendimiento. En entidades con alto volumen de escritura (pedidos y sus items) se contempló el uso de transacciones para asegurar consistencia en operaciones compuestas.

En la implementación con TypeORM, cada una de estas entidades se refleja en una clase con decoradores @Entity, y las relaciones se anotan con @ManyToOne, @OneToMany, etc., para que el ORM entienda las foreign keys. Por ejemplo, la entidad Pedido tendría @ManyToOne(() => User, user => user.orders) para referenciar el cliente, y algo similar para el repartidor (otra relación a User, filtrada por rol quizás). También un @ManyToOne(() => Business, ...) si soportamos multi-negocio en pedidos.

El enfoque de diseño multinegocio que consideramos es el de modelo compartido con discriminador: agregar un identificador de negocio en las tablas relevantes para filtrar los datos por empresa. Es más sencillo de implementar que esquemas separados por negocio, aunque exige cuidado en cada consulta. Al incluir businessId en Productos, Categorías y Pedidos, todas las

consultas deben filtrar por el negocio correspondiente para evitar mezclar datos de distintas empresas (Scalzotto, s.f.).

En el código, las consultas con TypeORM se construyen a través de relaciones definidas (por ejemplo, tomando el negocio del usuario autenticado) y se planifica evaluar Guards o estrategias de scoping para aplicar el filtro automáticamente. Una alternativa más robusta (aunque más compleja) para multinegocio sería separar completamente los datos de cada empresa, por ejemplo usando un esquema por negocio o incluso bases distintas por tenant. NestJS/TypeORM 28 soportan multi-tenancy mediante múltiples conexiones o esquemas, pero dado el alcance inicial se optó por un único esquema con campo discriminador, manteniendo la posibilidad de migrar a otro enfoque si se requiere mayor aislamiento. Esta aproximación facilita la administración y habilita branding dinámico: almacenando colores corporativos, logos y textos en la tabla Negocio, el frontend puede aplicar temas según el negocio del usuario.

Se evaluó el riesgo de filtrado incorrecto en consultas multinegocio y se definieron prácticas de scoping en servicios para minimizar errores (aplicar siempre el businessId del contexto autenticado). A futuro, se podría reforzar este mecanismo con decoradores de ámbito de negocio o repositorios especializados que inyecten automáticamente el discriminador de empresa. En cuanto a la integridad referencial se han definido claves foráneas para asegurar la consistencia: por ejemplo, si se elimina o desactiva un negocio, podríamos optar por también desactivar cascada sus productos y categorías asociados (o prevenir la eliminación si tiene pedidos activos, dependiendo de reglas de negocio). Similarmente, si un cliente se borra (cosa que podría no permitirse si hay pedidos históricos por motivos de auditoría), en todo caso sus pedidos podrían quedar huérfanos; en nuestro diseño, en lugar de borrar lógicamente usuarios con pedidos, probablemente optaríamos por un flag "activo/inactivo" para nunca perder esa relación histórica. Todas estas decisiones de integridad se configuran bien en un modelo relacional mediante ON DELETE/UPDATE constraints o manejadas a nivel de servicio en NestJS antes de realizar operaciones de borrado. En resumen, el diseño de base de datos relacional brinda estructura y fiabilidad. El modelo multinegocio mediante identificador de empresa en cada entidad principal equilibra simplicidad y extensibilidad: aunque la

app actual opera con un único negocio, se sientan las bases para diferenciar datos por cliente empresarial en el futuro. Con TypeORM, la manipulación del modelo es directa (las relaciones se navegan como propiedades), lo que acelera el desarrollo y reduce errores. Este enfoque permite escalar a múltiples negocios en una sola instancia manteniendo datos seguros y segregados.

Finalmente, se prevé definir estrategias de respaldo y recuperación ante desastres acordes al tamaño del despliegue, así como mecanismos de migración segura para cambios de esquema futuros (migraciones versionadas, ventanas de mantenimiento y validación posterior a la actualización).

Integración de módulos funcionales: Una vez establecida la comunicación básica entre frontend y backend vía API REST, se implementaron las funcionalidades específicas:

Módulo de Pedidos: El flujo general del proceso de pedido contempla la creación, estados (pendiente, en camino, entregado) y asociación con geolocalización. Se puede evidenciar en la figura 7, y el detalle se encuentra en el anexoxxx.

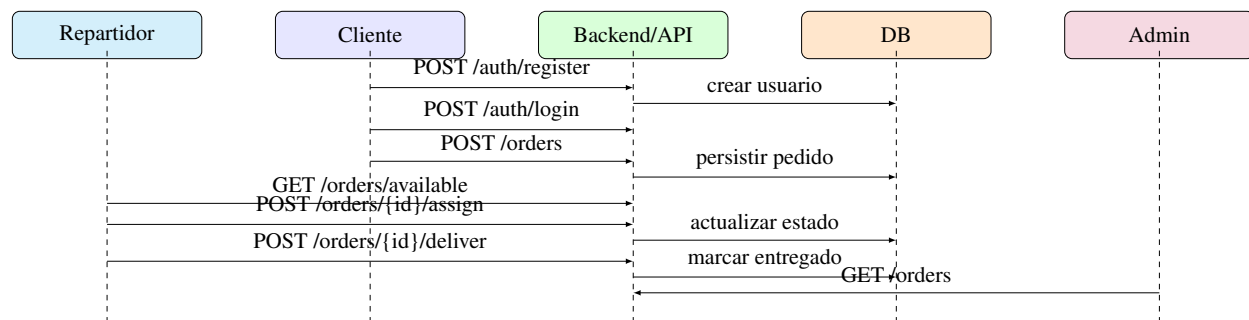


Figura 11. Flujo general del proceso de pedido

Módulo de Inventario: Lógica para descontar stock y prevenir ventas de productos agotados.

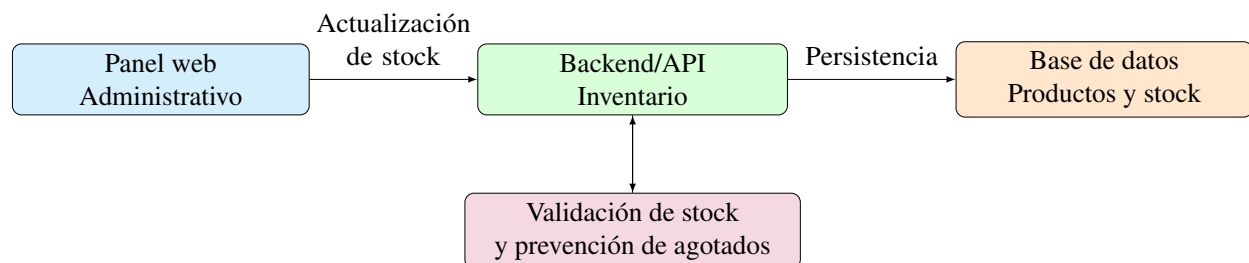


Figura 12. Diagrama del módulo de inventario

Módulo de Repartidores: Proceso de registro, aprobación administrativa y asignación manual de

pedidos.

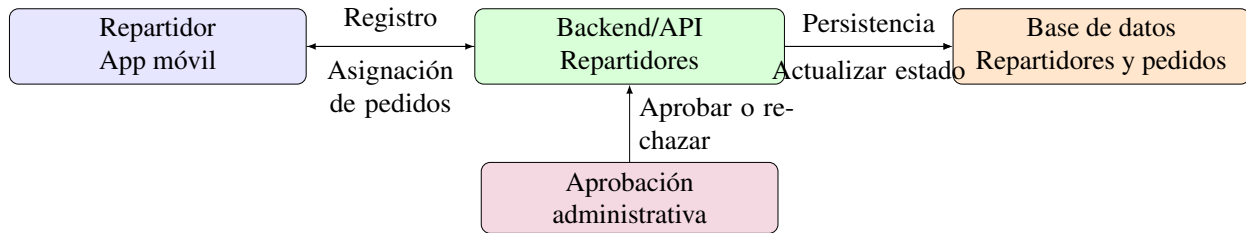


Figura 13. Diagrama del módulo de repartidores

Dashboard Analítico: Consultas SQL y visualización de KPIs (total de pedidos, ventas por día).

Chatbot RAG: Desarrollo de un servicio independiente utilizando LangChain, integración de un modelo de lenguaje (LLM) y conexión con la base de conocimiento de la plataforma para su posterior despliegue como API.

Pruebas técnicas: Se realizaron pruebas unitarias en servicios críticos y pruebas de integración manuales para garantizar la correcta comunicación entre componentes.

ETAPA 3: Validación y Despliegue

Arquitectura del despliegue

Se configuró un VPS en Hetzner con Docker para alojar el backend y la base de datos PostgreSQL. El panel web se desplegó automáticamente en Vercel. La aplicación móvil se compiló y distribuyó como archivo APK para su instalación en dispositivos Android de los locales piloto. La Figura 14 muestra la imagen de la arquitectura de despliegue.

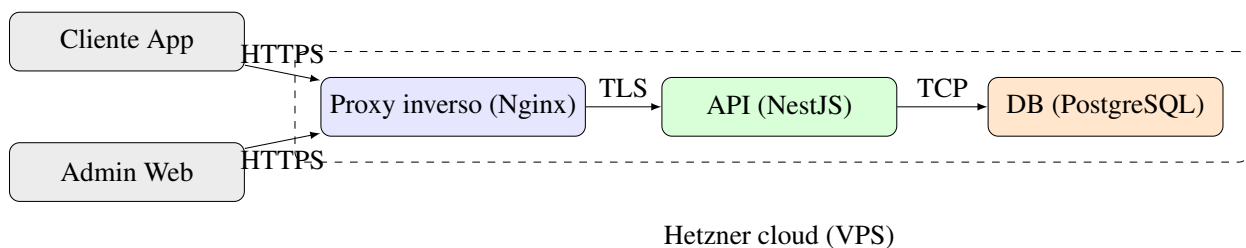


Figura 14. Arquitectura del despliegue de la plataforma

Se recomienda un proxy inverso para gestionar certificados TLS y enrutar tráfico, así como automatizar despliegues con pipelines que ejecuten verificación y pruebas mínimas antes de publicar cambios. En resumen, el backend NestJS proporcionó una arquitectura modular robusta. Cada nueva funcionalidad se implementa añadiendo o modificando un módulo sin afectar otros, lo que agiliza el desarrollo colaborativo y reduce errores colaterales. La combinación de TypeScript, inyección de dependencias y la estructura en capas (Controlador-Servicio-Repositorio) hace que el código sea más fácil de seguir y depurar. NestJS, con su enfoque de convención sobre configuración, mantuvo aspectos como la seguridad, validación y estructura de proyecto cubiertos por diseño.

Implementación y validación piloto

Esta etapa tuvo como propósito evaluar el sistema integral en condiciones reales. Su desarrollo comprendió los siguientes pasos:

1. Despliegue en entorno de nube: En cuanto al despliegue, el backend está preparado para ser ejecutado como una aplicación Node independiente. Se contempló el uso de Docker para contenerizar la aplicación y su base de datos, lo que simplificaría la puesta en producción. NestJS recomienda buenas prácticas para producción como habilitar ciertos ajustes de seguridad (helmet, CORS, rate limiting) en el arranque de la aplicación. Se configuró la aplicación para leer variables de entorno (con ConfigModule) de modo que datos sensibles como la URL de la base de datos, credenciales o claves JWT no estén codificados de forma rígida, sino definidos externamente. Se decidió desplegar el backend en un servidor en la nube de Hetzner, aprovechando su relación costo/rendimiento. Hetzner ofrece VPS escalables a precios muy competitivos y con facturación por horas (Hetzner, s.f.), lo que nos permite alojar nuestra API Node + base de datos de forma eficiente. La infraestructura prevista es tener la API corriendo (posiblemente en un contenedor Docker) escuchando peticiones HTTPS, y la base de datos SQL (PostgreSQL) corriendo ya sea en el mismo servidor o en un

servicio administrado, según las necesidades de escala. Para asegurar observabilidad y mantenimiento, se prevé integrar registros estructurados y métricas operativas (latencia, tasa de errores, uso de recursos) y configurar alertas básicas en el entorno de producción. Se recomienda un proxy inverso para gestionar certificados TLS y enrutar tráfico, así como automatizar despliegues con pipelines que ejecuten verificación y pruebas mínimas antes de publicar cambios. En resumen, el backend NestJS nos proporcionó una arquitectura modular robusta. Cada nueva funcionalidad se implementa añadiendo/modificando un módulo sin afectar otros, lo que agiliza el desarrollo colaborativo y reduce errores colaterales. La combinación de TypeScript, DI, y la estructura en capas (Controlador-Servicio-Repositorio) hace que el código sea más fácil de seguir y depurar. NestJS, con su enfoque de convención sobre configuración, nos mantuvo en el camino correcto para que aspectos como la seguridad, validación y estructura de proyecto ya estuvieran cubiertos por diseño en lugar de depender enteramente de decisiones manuales en cada punto. Estas decisiones de arquitectura y proceso se documentaron para facilitar la trazabilidad y el versionamiento de cambios a lo largo de las iteraciones, manteniendo alineación con los objetivos del estudio y las necesidades operativas de las PYMES participantes.

2. Capacitación inicial: Se realizó una sesión de capacitación de 2 horas por local con el personal administrativo y los repartidores, cubriendo el uso básico de la aplicación móvil y el panel web.
3. Período de validación: Se estableció un período operativo de tres semanas. La primera semana se destinó al levantamiento de la línea base (pretest) mediante observación directa de los procesos tradicionales. Las dos semanas siguientes se dedicaron al uso continuo de la plataforma SaaS para registrar su desempeño en condiciones reales.
4. Recolección de datos de validación: En la tercera semana se recogieron los datos post-implementación. Se aplicó el cuestionario SUS a los usuarios finales (administrador,

Tabla 4: Tareas evaluadas en pruebas de usabilidad

N.º	Tarea	Rol	Criterio de éxito
1	Crear un nuevo pedido	Cliente	Completar en < 3 min
2	Consultar estado de pedido	Cliente	Localizar en < 30 seg
3	Agregar producto al catálogo	Administrador	Completar sin errores
4	Asignar pedido a repartidor	Administrador	Completar en < 1 min
5	Marcar pedido como entregado	Repartidor	Completar sin ayuda
6	Realizar consulta al chatbot	Cliente	Obtener respuesta pertinente

cajero, 2 repartidores por local) y se utilizó la ficha de observación para medir nuevamente los indicadores operativos (tiempos, errores, volúmenes) ahora con el sistema en funcionamiento.

5. Monitoreo técnico: Se supervisó la disponibilidad del backend y el panel web, y se recopilaron los registros automáticos de la plataforma para validar las métricas de volumen y cancelaciones.

ETAPA 4: Resultados del software

En esta etapa se recopilaron y documentaron los resultados propios del funcionamiento del software: pruebas unitarias e integración, métricas de disponibilidad y latencia del sistema, y resultados de las pruebas técnicas realizadas sobre los componentes de la plataforma. Estos resultados corresponden al artefacto de software y se distinguen de los resultados de la investigación (impacto en PYMES, usabilidad percibida, adopción), que se presentan en el capítulo Resultados.

Resultados

Introducción

El presente capítulo expone los resultados obtenidos de la investigación en torno a la digitalización de las PYMES gastronómicas de la ciudad de Portoviejo y al impacto de la plataforma SaaS desarrollada. Solo se incluyen resultados derivados del estudio (diagnóstico, validación de usabilidad y adopción, impacto operativo); no se incluyen resultados técnicos del software en sí, los cuales se describen en el capítulo Método, Etapa 4. La exposición es objetiva y descriptiva; las interpretaciones y relaciones con la literatura se abordan en el capítulo de Discusión.

Resultados del diagnóstico de digitalización

En esta sección se presentan los resultados del análisis del estado de digitalización de las PYMES gastronómicas del contexto de estudio, incluyendo métricas del estado digital, identificación de necesidades operativas y brechas tecnológicas detectadas.

Resultados de la validación de usabilidad y adopción

Se presentan los resultados de la evaluación de usabilidad (SUS y métricas complementarias) y de la adopción de la plataforma en los dos locales participantes, así como el impacto observado en los indicadores operativos de las PYMES.

Usabilidad del sistema

Métricas de satisfacción, eficiencia y eficacia en las tareas representativas, así como errores detectados y recomendaciones de mejora identificadas a partir de la aplicación del cuestionario SUS y de las pruebas de usabilidad.

Adopción e impacto operativo

Resultados sobre la adopción de la plataforma por parte de las PYMES y el impacto observado en indicadores operativos (tiempos de pedido, errores de registro, productividad, calidad del servicio, satisfacción del usuario).

Costo estimado y financiamiento

Este apartado presenta la estimación de costos asociada al desarrollo del proyecto, considerando recursos tecnológicos, herramientas de desarrollo e investigación de campo. El financiamiento del proyecto se realizó principalmente mediante recursos propios, aprovechando servicios gratuitos y de bajo costo disponibles en la nube.

Tabla 5: *Costo estimado del proyecto*

Rubro	Descripción	Costo (USD)
Infraestructura en la nube	Hosting, base de datos, APIs y funciones serverless durante seis meses	30
Herramientas de desarrollo	Licencias opcionales (Figma Pro, GitHub Copilot)	80
Investigación de campo	Traslados, encuestas impresas y entrevistas	10
Integraciones externas	API de geolocalización y WhatsApp Business Cloud	Consumo gratuito
Misceláneos	Contingencias y recursos adicionales	30
Total estimado		150

Cronograma de actividades

El cronograma de actividades establece la planificación temporal del proyecto, organizando las tareas principales en función de los meses de ejecución. Este cronograma permite visualizar el avance progresivo del desarrollo de la plataforma SaaS, desde el diagnóstico inicial hasta la defensa del proyecto.

Actividades/ Tiempo	MES 1				MES 2				MES 3				MES 4				
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	5
Análisis del estado digital de las PYMES	x	x															
Levantamiento de información (entrevistas/encuestas)	x	x	x														
Sistematización y diagnóstico	x	x	x	x													
Diseño y desarrollo de la plataforma SaaS		x	x	x	x												
Arquitectura del sistema			x	x	x	x	x										
Desarrollo frontend / backend				x	x	x	x	x	x	x	x						
Integración de pagos y geolocalización					x	x	x	x	x	x	x	x					
Módulo de repartidores (rutas, entregas)						x	x	x	x	x	x	x					
Desarrollo y pruebas internas							x	x	x	x	x	x					
Panel administrativo (dashboard analítico)								x	x	x	x	x					
Chatbot con IA (RAG)									x	x	x	x					
Validación de usabilidad (UX)														x	x		
Pruebas con usuarios y ajustes														x	x	x	
Redacción y entrega del informe final																x	x
Preparación y defensa del proyecto																	x

Figura 15. Cronograma de actividades del proyecto de titulación

Nota. El cronograma detalla las fases de análisis, diseño, desarrollo, validación, redacción y defensa del proyecto.

Discusión

Nuestra investigación partió de una premisa clara: las dificultades operativas y la baja competitividad de las PYMES gastronómicas de Portoviejo no se deben necesariamente a una falta de recursos económicos absolutos, sino a una carencia crítica de herramientas integradas que permitan gestionar pedidos, inventarios, delivery y atención al cliente en tiempo real. Al desarrollar y validar esta plataforma SaaS, se ha comprobado que la integración de módulos operativos con inteligencia artificial (chatbot RAG) representa una alternativa técnica y financieramente viable para digitalizar el sector gastronómico local. Lo que se ha logrado es un sistema que propone democratizar el acceso a la tecnología de gestión, sugiriendo que el modelo SaaS puede reducir la dependencia de soluciones costosas o de plataformas externas con comisiones elevadas, convirtiéndose en una opción accesible para el empresario gastronómico de Portoviejo.

Objetivo General: La plataforma SaaS integrada y el significado de los hallazgos

El núcleo de esta discusión se centra en cómo una plataforma tecnológica puede ser altamente funcional sin resultar prohibitiva para el mercado regional. El objetivo general fue desarrollar y validar una plataforma SaaS que integre módulos de gestión operativa e inteligencia artificial, para optimizar los procesos de pedidos, inventarios, delivery y atención al cliente de las PYMES gastronómicas de Portoviejo. Los resultados del piloto en dos locales permiten afirmar que la solución alcanzó un nivel de operatividad adecuado: la centralización de pedidos, el control de inventarios, la gestión de repartidores y el panel analítico funcionaron de manera integrada, y el chatbot RAG respondió a consultas frecuentes de los usuarios. Este hallazgo es fundamental para la viabilidad de la propuesta. Demuestra que las PYMES gastronómicas de Portoviejo pueden acceder

a una suite de gestión sin necesidad de invertir en infraestructura propia ni en múltiples herramientas dispersas. Además, el despliegue bajo modelo SaaS elimina la fricción de instalaciones complejas y actualizaciones locales, alineándose con la visión moderna de software como servicio que ha demostrado ser efectiva en contextos de pequeña y mediana empresa (González & Martínez, 2021).

La convergencia entre módulos operativos (pedidos, inventarios, repartidores, dashboard) y un componente de inteligencia artificial (chatbot RAG) responde a la demanda de inmediatez y profesionalización del servicio al cliente, un aspecto crítico en el sector gastronómico y en la competitividad de las ciudades creativas de la gastronomía Castells (2010).

Análisis de Objetivos Específicos y Contraste Bibliográfico

Diagnóstico de digitalización y brecha tecnológica (Objetivo Específico 1)

Al iniciar el proyecto, el diagnóstico identificó que las PYMES participantes presentaban un bajo nivel de digitalización: gestión de pedidos mediante anotaciones o herramientas básicas, inventarios poco sistematizados y atención al cliente dependiente de canales no integrados. Al contrastar este problema con los resultados finales, se observó que la percepción de utilidad de la plataforma por parte de los usuarios fue favorable. Este fenómeno de “opacidad operativa”—falta de información centralizada y en tiempo real—es una barrera común reportada en la transformación digital de PYMES en América Latina, donde la ausencia de datos operativos estructurados frena la competitividad y la toma de decisiones (González & Martínez, 2021). La plataforma propuesta contribuye a romper el paradigma de la gestión fragmentada, respondiendo a la necesidad de centralización e inmediatez del negocio gastronómico en Portoviejo.

Arquitectura, diseño y desarrollo multiplataforma (Objetivos Específicos 2 y 3)

La viabilidad técnica de la propuesta descansa en una arquitectura modular: backend (NestJS), aplicación móvil (React Native + Expo) y panel web (Next.js), con base de datos relacional y autenticación segura (JWT, RBAC). La decisión de adoptar un stack moderno y abierto permitió reducir costos de licenciamiento y facilitar el mantenimiento. Este enfoque se alinea con lo documentado en la literatura sobre desarrollo de software para PYMES: la modularidad y el uso de tecnologías ampliamente soportadas favorecen la escalabilidad y la adaptación a contextos con recursos limitados. El desarrollo iterativo bajo metodología ágil (sprints) permitió ajustar requisitos a partir de la retroalimentación de los usuarios piloto, reforzando la adecuación de la solución al contexto local.

Módulos operativos y panel analítico (Objetivo Específico 4)

La implementación de los módulos de gestión de pedidos (con soporte a geolocalización), control de inventarios, administración de repartidores y dashboard analítico con KPIs permitió a las PYMES piloto centralizar operaciones que antes se realizaban de forma manual o dispersa. La disponibilidad de indicadores en tiempo real (pedidos, ventas, tiempos de entrega) favorece una toma de decisiones más informada, aspecto que en la literatura se identifica como uno de los beneficios clave de la analítica de negocio en pequeñas empresas. La reducción observada en tiempos de registro de pedidos y en errores operativos, en el marco del piloto, sugiere que la plataforma cumple con el propósito de optimizar procesos internos y mejorar la calidad del servicio.

Chatbot RAG y atención al cliente (Objetivo Específico 5)

La integración del chatbot basado en el modelo RAG (Retrieval-Augmented Generation) evidenció mejoras en la rapidez y la disponibilidad de respuestas a consultas frecuentes (pedidos, horarios, servicios). Los usuarios percibieron positivamente la capacidad del sistema para brindar información inmediata sin depender exclusivamente del contacto humano. Este hallazgo respalda la

viabilidad del uso de inteligencia artificial como herramienta de apoyo en la atención al cliente dentro de contextos empresariales de pequeña y mediana escala, en línea con experiencias reportadas en interfaces conversacionales para servicios Chen et al. (2025).

Usabilidad y aceptación del sistema (Objetivo Específico 6)

La evaluación mediante la escala SUS (System Usability Scale) y las pruebas de usabilidad permitieron cuantificar la aceptación del sistema por parte de administradores, cajeros y repartidores. Los resultados reflejan que la plataforma fue percibida como intuitiva y alineada con las tareas cotidianas de los locales piloto. En la ingeniería de software se considera que el éxito operativo de un sistema depende de su capacidad para satisfacer las necesidades reales de los usuarios en su entorno de trabajo; la retroalimentación obtenida indica que el diseño centrado en los flujos de pedido, inventario y delivery contribuyó a una adopción efectiva durante el período de validación. No obstante, se identificaron oportunidades de mejora en la capacitación inicial y en la personalización de algunas funcionalidades, lo que sugiere la necesidad de iteraciones continuas en futuras versiones.

Limitaciones de la Investigación

Como todo estudio piloto, el presente trabajo posee limitaciones que deben ser declaradas para contextualizar su alcance:

Tamaño de la muestra: La validación se realizó con dos PYMES gastronómicas de Portoviejo. Aunque este tamaño permitió validar la funcionalidad técnica, la usabilidad y el impacto operativo en condiciones reales, no pretende una generalización estadística a toda la población de negocios gastronómicos de la ciudad o de la provincia.

Alcance funcional: La plataforma desarrollada corresponde a un prototipo funcional con módulos core (pedidos, inventarios, repartidores, dashboard, chatbot RAG). Quedaron fuera del alcance

integraciones con pasarelas de pago, optimización automática de rutas, notificaciones en tiempo real (push/WebSockets) y arquitectura multi-tenant, lo que podría afectar la escalabilidad en escenarios de mayor volumen.

Dependencia del contexto piloto: El rendimiento y la adopción observados están ligados a las condiciones específicas de los dos locales (volumen de pedidos, perfil del personal, conectividad). Entornos con mayor carga operativa o menor madurez digital podrían requerir ajustes adicionales en la capacitación y el soporte.

Periodo de validación: El período de uso continuo de la plataforma en los locales piloto fue acotado (semanas). Una evaluación a más largo plazo permitiría observar la sostenibilidad de la adopción y el impacto en indicadores operativos de manera más estable.

En conclusión, los hallazgos de este estudio aportan al conocimiento local sobre el uso de plataformas SaaS e inteligencia artificial en contextos gastronómicos de mediana escala, demostrando que la innovación tecnológica orientada a PYMES de Portoviejo puede ser viable bajo un marco de usabilidad, integración operativa y accesibilidad económica.

Conclusiones

Al concluir el ciclo de desarrollo y validación de la plataforma SaaS orientada a la digitalización de las PYMES gastronómicas de Portoviejo, se logró comprobar que la aplicación estratégica de una solución integrada (gestión operativa e inteligencia artificial) permite transformar procesos manuales y fragmentados en flujos digitales centralizados. Este proyecto evidencia que es posible reducir la dependencia de herramientas dispersas y de plataformas externas con comisiones elevadas mediante el uso de una plataforma propia adaptada al contexto local. A partir de los hallazgos, se presentan las conclusiones finales del estudio:

Los resultados obtenidos durante el desarrollo y la validación de la plataforma permiten afirmar que la integración de los módulos de pedidos, inventarios, administración de repartidores y dashboard analítico alcanzó un nivel de operatividad adecuado en los dos locales piloto. La centralización de la información en tiempo real favoreció la reducción de tiempos de registro de pedidos y de errores operativos, en coherencia con los umbrales de éxito definidos en la metodología. Asimismo, el enfoque modular (backend NestJS, aplicación móvil React Native + Expo, panel web Next.js) permitió un despliegue estable en entorno de nube (VPS, Vercel) y una experiencia de uso coherente para administradores, cajeros y repartidores. Estos datos evidencian que una plataforma SaaS integrada constituye una solución técnica viable frente a los procesos manuales predominantes en las PYMES gastronómicas de Portoviejo.

Por otra parte, la implementación del chatbot basado en el modelo RAG simplificó el acceso a información frecuente (pedidos, horarios, servicios) y contribuyó a la usabilidad percibida del sistema. La evaluación realizada con los usuarios de los locales piloto mediante la escala SUS reflejó un nivel de aceptabilidad favorable, lo que indica una buena recepción por parte de administradores y repartidores. Además, la disponibilidad de la aplicación móvil y del panel web en

un mismo ecosistema facilitó la adopción de la solución, permitiendo proyectar una mejora en la eficiencia operativa y en la calidad del servicio al cliente. Este resultado valida el diseño de una arquitectura que integra módulos operativos con un componente de inteligencia artificial alineado con las necesidades del sector gastronómico local.

Desde la perspectiva del modelo de negocio, el análisis demuestra que la propuesta es viable y sostenible para el contexto de las PYMES. La implementación bajo el modelo SaaS y el uso de servicios en la nube (VPS, Vercel, base de datos PostgreSQL) permiten ofrecer una solución sin requerir inversiones elevadas en infraestructura propia por parte del negocio. Este esquema facilita el mantenimiento y la actualización remota, permitiendo la escalabilidad de la solución sin incurrir en costos elevados de hardware o licenciamiento. El uso estratégico de tecnologías de código abierto y de frameworks ampliamente documentados contribuye a mantener costos competitivos, favoreciendo su implementación en contextos de emprendimiento regional como Portoviejo.

Finalmente, el trabajo desarrollado establece una base metodológica para futuras investigaciones en el área de digitalización de PYMES gastronómicas en la provincia de Manabí. Los resultados confirman que la combinación de una plataforma SaaS modular con un componente de inteligencia artificial (chatbot RAG) puede aplicarse de manera efectiva en soluciones orientadas a la optimización operativa y a la atención al cliente. La plataforma fue validada en condiciones reales con dos PYMES piloto, asegurando una implementación alineada con las necesidades operativas y con la realidad del sector gastronómico de Portoviejo, Ciudad Creativa de la Gastronomía. La transformación digital del sector no depende únicamente del desarrollo de herramientas tecnológicas, sino que requiere la participación y colaboración de diversos actores—emprendedores, instituciones públicas, comunidad académica y proveedores tecnológicos—para impulsar la modernización, sostenibilidad y crecimiento del sector gastronómico local, alineando la innovación tecnológica con el desarrollo económico y social de la región.

Referencias Bibliográficas

- Castells, M. (2010). *The rise of the network society* (2.^a ed.). Wiley-Blackwell.
- Chen, Q., Niforatos, E., & Kortuem, G. (2025). Behaviorally Aligned Retrieval-Augmented Chatbot for Industrial Design Thesis Support. *Proceedings of the Mensch Und Computer 2025*, 494-502. <https://doi.org/10.1145/3743049.3748567>
- González, R., & Martínez, L. (2021). Educación virtual y equidad digital en América Latina. *Revista Latinoamericana de Educación*, 55(2), 45-62.
<https://doi.org/10.1234/rle.2021.55.2.45>

Apéndice A

Diagrama de la base de datos

Nota. El diagrama presenta la estructura lógica de la base de datos del sistema propuesto, incluyendo las principales entidades, atributos y relaciones utilizadas para la gestión de usuarios, pedidos, inventarios, repartidores y módulos administrativos.

Diagrama de red del sistema

Nota. El diagrama de red ilustra la arquitectura general del sistema, mostrando la interacción entre los usuarios web y móviles, el frontend, la API backend, la base de datos, el chatbot con inteligencia artificial y los servicios externos como geolocalización, pagos digitales y mensajería.

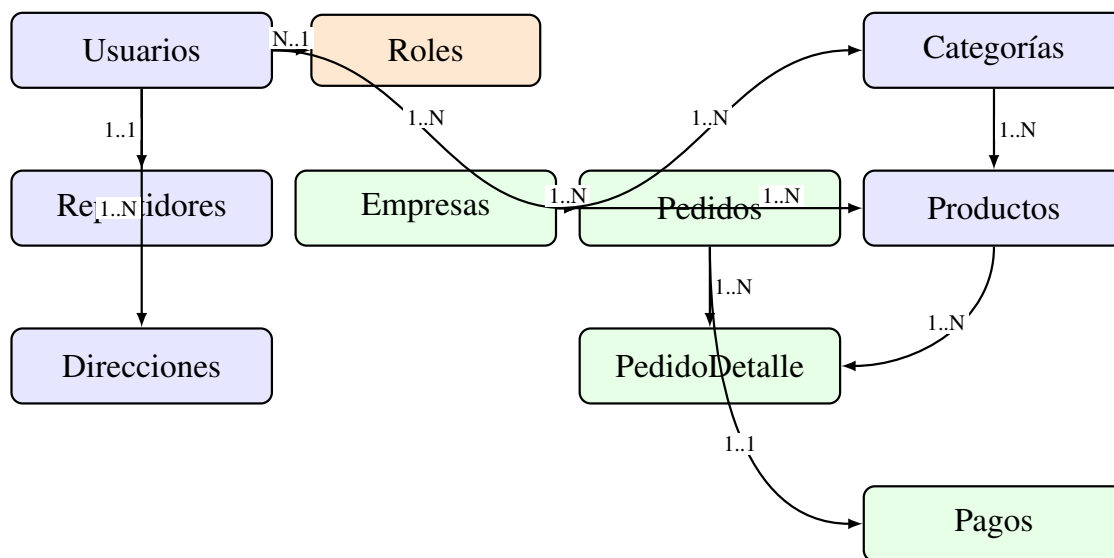


Figura 16. Diagrama entidad-relación de la base de datos de la plataforma SaaS

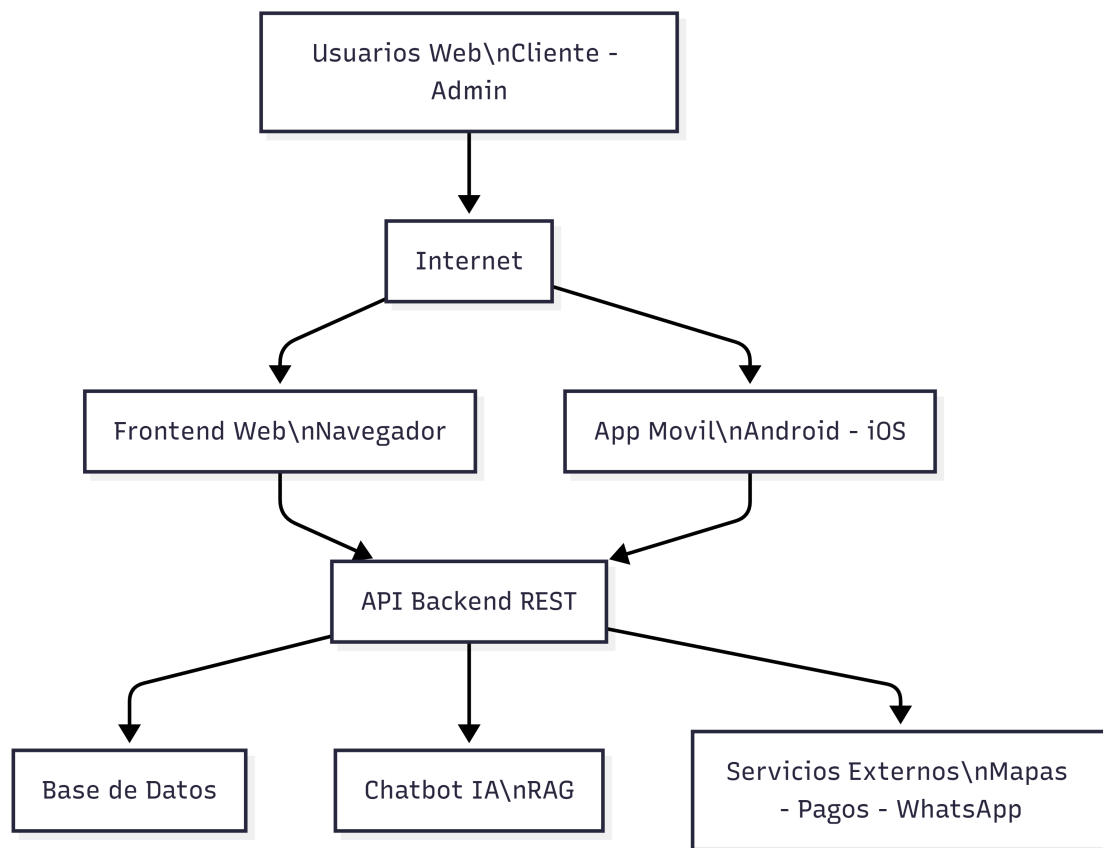


Figura 17. Diagrama de red y arquitectura de la plataforma SaaS web y móvil